



UNIVERSITÀ DEGLI STUDI DI NAPOLI
FEDERICO II

Università degli Studi di Napoli Federico II
Corso di Ingegneria del Software — Canale 2

BugBoard26

Documento di Specifica dei Requisiti Software

Sistema distribuito per issue tracking, collaborazione e reporting

Autori

Alessandro Minopoli n86004964

Simone Mollo n86004971

Stack di riferimento React 18 • Spring Boot 3.3 • PostgreSQL • Docker • JWT

Versione: 2.0

Data: 19 febbraio 2026

Stato: Consegna progetto completo

Anno Accademico 2025/2026

Indice

1	Introduzione	4
1.1	Scopo del documento	4
1.2	Descrizione del sistema	4
1.3	Riferimenti	4
2	Glossario	4
3	Individuazione e caratterizzazione degli utenti (attori)	7
4	Modellazione dei Casi d'Uso	8
4.1	Tabella dei Casi d'Uso per Attore	8
5	Tabella dei Casi d'Uso	8
5.1	Descrizione dettagliata dei casi d'uso	8
5.1.1	Utente non registrato	9
5.1.2	Utente registrato (USER)	9
5.1.3	Utente ReadOnly	10
5.1.4	Admin	10
5.1.5	Sistema	10
5.2	Diagrammi UML dei Casi d'Uso	11
5.2.1	Diagramma Generale del Sistema	12
5.2.2	Diagramma Dettagliato — Gestione Bug	13
5.2.3	Diagramma Dettagliato — Amministrazione	14
5.3	Convenzioni grafiche	14
6	Formalizzazione dei Casi d'Uso (Cockburn)	15
6.1	Caso d'Uso 1: Creazione nuovo bug	15
6.2	Caso d'Uso 2: Assegnazione bug a un utente	16
6.3	Caso d'Uso 3: Aggiornamento stato di un bug assegnato	17
6.4	Caso d'Uso 4: Consultazione report mensile (Admin)	18
7	Requisiti Non Funzionali e di Sistema	19
7.1	Usabilità	19
7.2	Prestazioni	19
7.3	Sicurezza	19
7.4	Affidabilità e Disponibilità	20
7.5	Manutenibilità e Portabilità	20
7.6	Vincoli di Sistema	20
8	Prototipazione visuale attraverso Mock-up	21
8.1	Scelte Progettuali	21
8.1.1	Layout e Navigazione	21
8.1.2	Palette Colori e Badge	21
8.1.3	Componenti UI riutilizzati	21
8.2	Mock-up delle interfacce utente	21
8.2.1	Mock-up 1 — Pagina di Login	22
8.2.2	Mock-up 2 — Pagina Informativa Richiesta Account	22
8.2.3	Mock-up 3 — Home / Dashboard	23
8.2.4	Mock-up 4 — Lista Bug con Filtri	23
8.2.5	Mock-up 5 — Dettaglio Bug	24

8.2.6	Mock-up 6 — Form Creazione/Modifica Bug [UC01]	24
8.2.7	Mock-up 7 — Assegnazione Bug [UC02]	25
8.2.8	Mock-up 8 — Notifiche	26
8.2.9	Mock-up 9 — Profilo Utente	26
8.2.10	Mock-up 10 — Report Mensile [UC04]	27
8.2.11	Mock-up 11 — Gestione Utenti (Admin)	27
8.2.12	Mock-up 12 — Archivio Bug	28
8.2.13	Mock-up 13 — Export Dati	28
8.3	Osservazioni emerse dalla prototipazione	29
9	Design del Sistema	29
9.1	Descrizione dell'architettura	29
9.1.1	Deployment Diagram	29
9.1.2	Component Diagram	30
9.1.3	Diagrammi di approfondimento architetturale	30
9.2	Criteri di design adottati	34
9.3	Scelte tecnologiche	34
9.3.1	Frontend	34
9.3.2	Backend	35
9.3.3	Testing	35
9.3.4	DevOps e Deployment	35
9.4	Comunicazione tra i livelli	36
9.5	Sicurezza	36
10	Design del Software	36
10.1	Schema per la persistenza dati	36
10.1.1	Tabella <code>users</code>	37
10.1.2	Tabella <code>bugs</code>	37
10.1.3	Tabella <code>comments</code>	37
10.1.4	Tabella <code>history</code>	38
10.1.5	Tabella <code>labels</code>	38
10.1.6	Tabella di join <code>bug_labels</code>	38
10.1.7	Tabella <code>notifications</code>	38
10.1.8	Collection table <code>bug_attachments</code>	39
10.1.9	Diagramma E-R semplificato	39
10.1.10	Diagramma di persistenza dettagliato	40
10.2	Diagramma delle classi di design	41
10.3	Diagramma comportamentale — Sequence Diagram	42
10.4	Scelte di software design	44
10.4.1	Pattern Repository	44
10.4.2	Pattern Service Layer	44
10.4.3	Record Java per i DTO	44
10.4.4	Specification Pattern per le query	44
10.4.5	Audit Trail tramite History	45
10.4.6	Autenticazione stateless con JWT	45
10.4.7	Gestione delle autorizzazioni	45
10.5	Evidenza dell'uso di strumenti di versioning	45

11 Test Plan – Gestione Bug	46
11.1 Obiettivo	46
11.2 Funzionalità sotto test	46
11.3 Approccio di testing	47
11.4 Casi di test pianificati	47
11.4.1 create(BugCreateRequest, UUID)	47
11.4.2 assign(UUID, AssignRequest, UUID)	47
11.4.3 patch(UUID, BugPatchRequest, UUID, boolean)	47
11.5 Criteri di ingresso e uscita	47
11.6 Ambiente di esecuzione	48
12 Strategie di Testing	48
12.1 Metodi testati	48
12.2 Classi di equivalenza	48
12.2.1 create(BugCreateRequest, UUID)	48
12.2.2 assign(UUID, AssignRequest, UUID)	48
12.2.3 patch(UUID, BugPatchRequest, UUID, boolean)	49
12.3 Analisi dei Valori Limite (Boundary Value Analysis)	49
12.3.1 create(BugCreateRequest, UUID)	49
12.3.2 assign(UUID, AssignRequest, UUID)	49
12.3.3 patch(UUID, BugPatchRequest, UUID, boolean)	50
12.4 Copertura strutturale	50
12.5 Tecnica di isolamento: Mock Objects	51
12.6 Riepilogo dei risultati	51
13 Report di Qualità del Codice (Backend)	51
13.1 Strumenti di analisi utilizzati	51
13.2 Dashboard di qualità	52
13.3 Metriche di dimensione	52
13.4 Distribuzione del codice per package	52
13.5 Issue rilevate (dettaglio)	53
13.5.1 Bugs (0 issue — Rating A)	53
13.5.2 Code Smells (24 issue — Rating A)	53
13.5.3 Security Hotspot (4 issue — Rating B)	53
13.6 Duplicazioni	53
13.7 Punti di forza	53
13.8 Riepilogo Quality Gate	54
14 Valutazione dell’Usabilità	54
14.1 Metodologia	54
14.1.1 Partecipanti	54
14.1.2 Ambiente di test	54
14.1.3 Protocollo	54
14.2 Risultati dei Task	55
14.3 Ispezione Euristica (Nielsen)	55
14.4 Questionario SUS	55
14.4.1 Risposte individuali	56
14.4.2 Calcolo punteggi SUS	56
14.5 Feedback qualitativo (Debriefing)	56
14.6 Azioni correttive	57

1 Introduzione

1.1 Scopo del documento

Il presente documento costituisce la Specifica dei Requisiti Software (SRS) per il sistema **BugBoard26**, sviluppato nell'ambito del corso di Ingegneria del Software. Il documento descrive le funzionalità del sistema, i suoi attori, i casi d'uso formalizzati, i requisiti non funzionali e fornisce una prima prototipazione visuale dell'interfaccia utente.

1.2 Descrizione del sistema

BugBoard26 è un'applicazione web per il **tracciamento e la gestione collaborativa di segnalazioni di bug**. Il sistema consente a team di sviluppo di:

- segnalare bug, feature request e miglioramenti;
- assegnare le segnalazioni agli sviluppatori;
- tracciare il ciclo di vita di ogni segnalazione (apertura, presa in carico, risoluzione, archiviazione);
- aggiungere commenti e allegati fotografici;
- visualizzare statistiche e generare report mensili sull'attività del team.

Sintesi del documento. Questo PDF raccoglie in modo integrato requisiti, casi d'uso, prototipi, design architetturale, strategie di testing e indicatori di qualità del progetto **BugBoard26**, mantenendo coerenza con il codice e gli artefatti presenti nel repository.

1.3 Riferimenti

- A. Cockburn, *Writing Effective Use Cases*, Addison-Wesley, 2001
- Traccia progetto corso Ingegneria del Software — Canale 2

2 Glossario

Di seguito sono riportati i termini tecnici e di dominio utilizzati nel presente documento e nel sistema BugBoard26.

Admin	Utente con privilegi amministrativi completi. Può gestire utenti, accedere a tutti i bug del sistema, generare report ed esportare i dati.
Allegato	File immagine (JPEG, PNG, GIF, WebP, dimensione massima 5 MB) caricato in associazione a un bug per documentarne visivamente il problema o la soluzione.
Archiviazione	Operazione che sposta un bug in stato <i>Archived</i> , rimuovendolo dalla lista attiva senza eliminarlo definitivamente dal sistema.
Assegnatario	Utente registrato responsabile della risoluzione di un determinato bug.
Bug	Segnalazione di un comportamento anomalo, errato o inatteso di un sistema software. Nel contesto di BugBoard26 il termine indica genericamente qualsiasi segnalazione inserita nel sistema, indipendentemente dal tipo (Question, Bug, Documentation, Feature).

BugBoard26	Il sistema software oggetto di questo documento. Applicazione web per il tracciamento e la gestione collaborativa delle segnalazioni di bug.
Caso d'uso	Descrizione di un'interazione tra un attore e il sistema per raggiungere un obiettivo specifico. Formalizzato secondo il metodo di Alistair Cockburn.
Commento	Testo inserito da un utente in associazione a un bug per fornire aggiornamenti, chiarimenti o informazioni aggiuntive.
Cookie HttpOnly	Cookie accessibile dal browser ma non leggibile direttamente da script JavaScript lato client. In BugBoard26 viene usato per trasportare il token JWT in modo più sicuro.
CORS (Cross-Origin Resource Sharing)	Meccanismo di sicurezza del browser che regola quali origini possono inviare richieste HTTP a un server diverso dal proprio dominio/porta.
CSRF (Cross-Site Request Forgery)	Attacco in cui un sito malevolo induce il browser dell'utente a inviare richieste non volute a un altro servizio autenticato.
Dashboard	Schermata iniziale dell'applicazione dopo il login, che mostra metriche riassuntive e scorciatoie verso le funzionalità principali.
Database relazionale	Sistema di persistenza che organizza i dati in tabelle collegate tra loro tramite chiavi primarie e chiavi esterne.
Deployment	Fase di rilascio e pubblicazione del software in un ambiente eseguibile, ad esempio su cloud o tramite container.
DTO (Data Transfer Object)	Struttura dati usata per trasferire informazioni tra frontend e backend o tra livelli applicativi, evitando di esporre direttamente le entità interne.
Docker	Piattaforma di containerizzazione utilizzata per impacchettare e distribuire i componenti dell'applicazione (frontend e backend) in modo indipendente dall'ambiente host.
Documentation	Tipo di segnalazione che indica un problema o una richiesta di correzione relativa alla documentazione del progetto o del prodotto.
Export	Funzionalità che consente di scaricare i dati del sistema in formati esterni come CSV, Excel o PDF.
Feature	Tipo di segnalazione che indica la richiesta di implementazione di una nuova funzionalità nel sistema.
Figma	Strumento di progettazione UI/UX usato dal team per realizzare wireframe e mock-up delle schermate prima dell'implementazione React.
Frontend	Parte dell'applicazione con cui interagisce direttamente l'utente, sviluppata in questo progetto con React, TypeScript e Vite.
H2	Database relazionale leggero ed embeddable usato nell'ambiente di sviluppo locale per avviare rapidamente il backend senza installazioni esterne.

History (Storico)	Registro cronologico di tutte le modifiche apportate a un bug nel tempo: cambi di stato, assegnazioni, commenti, aggiunta di allegati.
Hibernate	Implementazione ORM integrata in Spring Data JPA che traduce operazioni su oggetti Java in operazioni sul database relazionale.
JWT (JSON Web Token)	Standard aperto (RFC 7519) utilizzato per autenticare in modo sicuro le richieste HTTP tra frontend e backend. Il token viene generato al login e incluso in ogni richiesta successiva.
KPI (Key Performance Indicator)	Indicatore sintetico usato per monitorare l'andamento del sistema, ad esempio numero di bug aperti o tempo medio di risoluzione.
Label	Etichetta testuale opzionale associabile a un bug per categorizzarlo ulteriormente (es. "frontend", "performance", "regression").
Mock object	Oggetto simulato usato nei test per sostituire dipendenze reali come repository o servizi esterni, permettendo di verificare la sola logica del componente sotto test.
OpenAPI / Swagger	Standard e relativa interfaccia usati per documentare e consultare gli endpoint REST esposti dal backend.
ORM (Object-Relational Mapping)	Tecnica che collega classi e oggetti del linguaggio di programmazione alle tabelle del database.
Priorità	Livello di urgenza opzionale di un bug. I valori previsti sono: <i>Low</i> , <i>Medium</i> , <i>High</i> , <i>Urgent</i> .
ReadOnly (Utente)	Ruolo con accesso in sola lettura. L'utente ReadOnly può visualizzare bug, commenti e allegati, ma non può creare, modificare o commentare.
Repository	Componente del backend incaricato dell'accesso ai dati persistiti. In Spring Data JPA viene tipicamente modellato come interfaccia dedicata a una certa entità.
Report Mensile	Vista amministrativa relativa a un mese selezionato che riassume numero di bug aperti, gestiti e risolti, tempo medio di risoluzione e dettaglio per utente; i dati possono essere successivamente esportati nei formati supportati dal sistema.
REST API	Insieme di endpoint HTTP che permettono la comunicazione tra frontend e backend tramite richieste come GET, POST, PATCH o DELETE.
Ruolo	Insieme di permessi associato a un utente. I ruoli previsti sono: <i>USER</i> (utente standard), <i>READONLY</i> (sola lettura), <i>ADMIN</i> (amministratore).
SonarQube	Strumento di analisi statica del codice usato per misurare qualità, vulnerabilità, duplicazioni e code smell del backend.
Spring Boot	Framework Java usato per sviluppare il backend, configurare l'applicazione e pubblicare rapidamente servizi REST.

Stato (Status)	Fase del ciclo di vita di un bug. Gli stati previsti sono: • <i>Todo</i>: bug appena creato, non ancora preso in carico • <i>In Progress</i>: bug in fase di risoluzione • <i>Resolved</i>: bug risolto • <i>Archived</i>: bug archiviato
Tipo (Type)	Categoria della segnalazione. I valori previsti sono: <i>Question</i> , <i>Bug</i> , <i>Documentation</i> , <i>Feature</i> .
UI (User Interface)	Insieme degli elementi grafici con cui l'utente interagisce, come bottoni, form, card, badge e pagine.
USER (Utente registrato)	Ruolo standard. L'utente può creare bug, commentare, caricare allegati, visualizzare i bug assegnati a lui e consultare il proprio profilo/account.
Vite	Build tool moderno usato nel frontend per avvio rapido in sviluppo, bundling e gestione del proxy verso il backend.
WCAG	Linee guida internazionali per l'accessibilità dei contenuti web, utili per valutare contrasto, leggibilità e usabilità dell'interfaccia.

3 Individuazione e caratterizzazione degli utenti (attori)

Il sistema prevede quattro tipologie di attori umani e un attore di sistema:

Utente non registrato (Guest)

Chiunque acceda all'applicazione senza credenziali. Può accedere alla pagina di login e alla pagina informativa per richiesta account. Non ha accesso ad alcuna funzionalità interna.

Utente registrato (USER)

Utente autenticato con ruolo standard. Può creare e modificare bug, aggiungere commenti e allegati, visualizzare lo storico e consultare il proprio profilo.

Utente ReadOnly

Utente autenticato con accesso in sola lettura. Può visualizzare bug, commenti e allegati, ma non può eseguire operazioni di scrittura.

Admin

Utente con privilegi amministrativi completi. Eredita tutte le funzionalità di USER e può in aggiunta: gestire gli utenti del sistema, assegnare bug, visualizzare metriche, generare report, esportare dati.

Sistema

Attore non umano che esegue operazioni automatiche: invio di notifiche, generazione dello storico modifiche, validazione degli allegati.

4 Modellazione dei Casi d'Uso

4.1 Tabella dei Casi d'Uso per Attore

5 Tabella dei Casi d'Uso

Attore	Casi d'uso
Utente non registrato	• Accesso alla pagina informativa per richiesta account • Accesso alla pagina di login
Utente registrato (USER)	• Login/logout al sistema • Visualizzazione lista bug con filtri (tipo, stato, priorità, label) • Visualizzazione dettagli singolo bug • Creazione nuovo bug • Modifica bug assegnati all'utente • Aggiunta commenti ai bug • Upload allegati immagini ai bug • Visualizzazione bug assegnati all'utente • Modifica stato dei bug assegnati • Visualizzazione storico modifiche bug • Visualizzazione notifiche in-app • Visualizzazione profilo personale • Esportazione dati in formato CSV, Excel e PDF
Utente ReadOnly	• Login al sistema • Visualizzazione lista bug con filtri • Visualizzazione dettagli singolo bug • Visualizzazione commenti e allegati • Visualizzazione storico modifiche • Visualizzazione notifiche in-app • Visualizzazione profilo proprio (sola lettura) • Esportazione dati in formato CSV, Excel e PDF
Admin	• Tutti i casi d'uso dell'Utente registrato • Gestione utenti (visualizzazione e creazione account con ruolo) • Visualizzazione metriche di sistema • Consultazione report mensili • Accesso completo a tutti i bug del sistema • Gestione assegnazioni dei bug • Marcatura bug come duplicati • Archiviazione manuale e gestione scadenze dei bug
Sistema	• Invio notifiche automatiche in-app su cambiamenti bug • Archiviazione automatica bug risolti dopo periodo di inattività • Generazione automatica storico modifiche • Validazione automatica allegati (tipo e dimensione)

Tabella 1: Tabella completa dei casi d'uso per BugBoard26

5.1 Descrizione dettagliata dei casi d'uso

5.1.1 Utente non registrato

- **Visualizzazione informazioni pubbliche:** Accesso alla pagina informativa pubblica del progetto, con presentazione sintetica del servizio e indicazioni su come richiedere un account.
- **Accesso alla pagina informativa per richiesta account:** Consultazione della pagina "Registrati", che chiarisce che la creazione degli account è gestita dall'amministratore e non avviene in self-service.
- **Accesso alla pagina di login:** Possibilità di accedere al sistema utilizzando credenziali esistenti.

5.1.2 Utente registrato (USER)

- **Login/logout al sistema:** Autenticazione tramite credenziali email/password con generazione di token JWT per sessioni sicure.
- **Visualizzazione lista bug con filtri:** Consultazione dell'elenco completo dei bug con possibilità di filtrare per tipo, stato, priorità, label e ricerca testuale.
- **Visualizzazione dettagli singolo bug:** Accesso completo alle informazioni di un bug specifico, inclusi descrizione, commenti, allegati e storico modifiche.
- **Creazione nuovo bug:** Inserimento di una nuova segnalazione con descrizione dettagliata, priorità opzionale, tipo e label appropriate.
- **Modifica bug assegnati all'utente:** Possibilità di aggiornare informazioni e stato dei bug assegnati al proprio account; gli Admin possono modificare qualsiasi bug.
- **Aggiunta commenti ai bug:** Inserimento di commenti testuali per fornire aggiornamenti, soluzioni o richiedere chiarimenti.
- **Upload allegati immagini ai bug:** Caricamento di immagini (JPEG, PNG, GIF, WebP) fino a 5MB per documentare visivamente il problema.
- **Visualizzazione bug assegnati all'utente:** Consultazione rapida delle issue assegnate al proprio account tramite sezione dedicata.
- **Modifica stato dei bug assegnati:** L'assegnatario può aggiornare lo stato del bug lungo il workflow TODO / IN_PROGRESS / RESOLVED.
- **Visualizzazione storico modifiche bug:** Consultazione cronologica di tutte le modifiche apportate a un bug nel tempo.
- **Visualizzazione notifiche in-app:** Consultazione delle notifiche ricevute per assegnazioni, commenti e aggiornamenti rilevanti sui bug.
- **Visualizzazione profilo personale:** Accesso ai dati del proprio account e alle scorciatoie verso le funzionalità disponibili.
- **Esportazione dati:** Download dell'elenco bug nei formati CSV, Excel e PDF.

5.1.3 Utente ReadOnly

- **Login al sistema:** Accesso con credenziali per utenti con privilegi di sola lettura.
- **Visualizzazione lista bug con filtri:** Consultazione dell'elenco bug con possibilità di applicare filtri di ricerca.
- **Visualizzazione dettagli singolo bug:** Accesso in sola lettura alle informazioni complete di un bug.
- **Visualizzazione commenti e allegati:** Lettura di tutti i commenti e download/visualizzazione degli allegati associati.
- **Visualizzazione storico modifiche:** Consultazione della cronologia delle modifiche senza possibilità di intervento.
- **Visualizzazione notifiche in-app:** Consultazione del centro notifiche senza possibilità di modificare i bug.
- **Visualizzazione profilo proprio:** Accesso limitato alle informazioni del proprio profilo utente.
- **Esportazione dati:** Download dei dati in formato CSV, Excel o PDF anche in modalità sola lettura.

5.1.4 Admin

- **Tutti i casi d'uso dell'Utente registrato:** L'amministratore eredita tutte le funzionalità dell'utente standard.
- **Gestione utenti:** Creazione di nuovi account e assegnazione del ruolo appropriato agli utenti del sistema.
- **Visualizzazione metriche di sistema:** Accesso a dashboard con statistiche, grafici e KPI del sistema.
- **Consultazione report mensili:** Accesso al riepilogo mensile delle attività del sistema per il periodo selezionato, con possibilità di passare poi alla sezione export.
- **Accesso completo a tutti i bug:** Possibilità di visualizzare e modificare qualsiasi bug del sistema indipendentemente dall'assegnazione.
- **Gestione assegnazioni:** Possibilità di assegnare o riassegnare un bug a un membro del team, con suggerimento dell'utente meno carico.
- **Marcatura bug come duplicati:** Collegamento di un bug a un altro esistente, con archiviazione automatica del duplicato.
- **Archiviazione manuale e gestione scadenze:** Possibilità di archiviare manualmente un bug e impostare deadline opzionali in creazione o modifica.

5.1.5 Sistema

- **Invio notifiche automatiche:** Meccanismo automatico di notifica in-app per aggiornamenti sui bug che riguardano l'utente.
- **Archiviazione automatica:** Processo schedato per archiviare automaticamente bug inattivi dopo un periodo configurabile.

- **Generazione automatica storico:** Registrazione automatica di ogni modifica apportata ai bug nel database.
- **Validazione automatica allegati:** Controllo automatico di tipo, dimensione e sicurezza dei file caricati.

5.2 Diagrammi UML dei Casi d'Uso

5.2.1 Diagramma Generale del Sistema

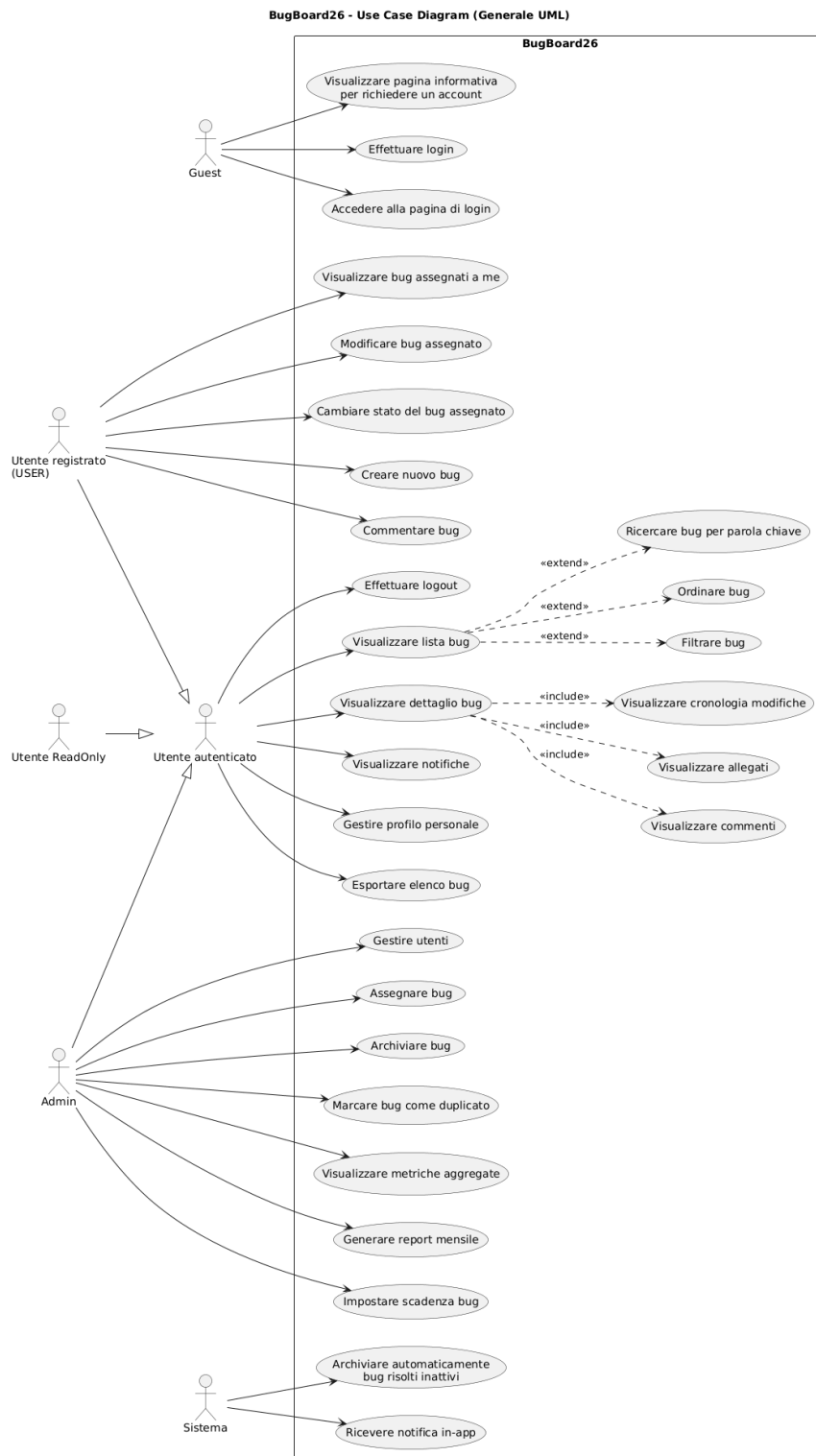


Figura 1: Diagramma generale dei casi d'uso — BugBoard26

5.2.2 Diagramma Dettagliato — Gestione Bug

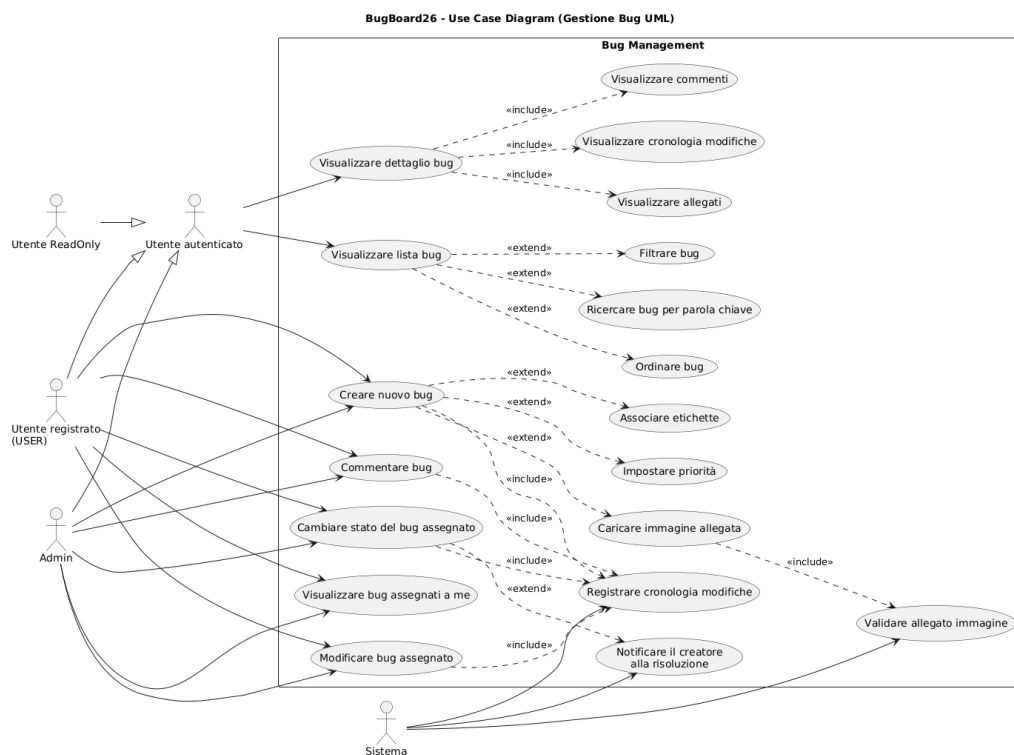


Figura 2: Diagramma dettagliato — Gestione Bug

5.2.3 Diagramma Dettagliato — Amministrazione

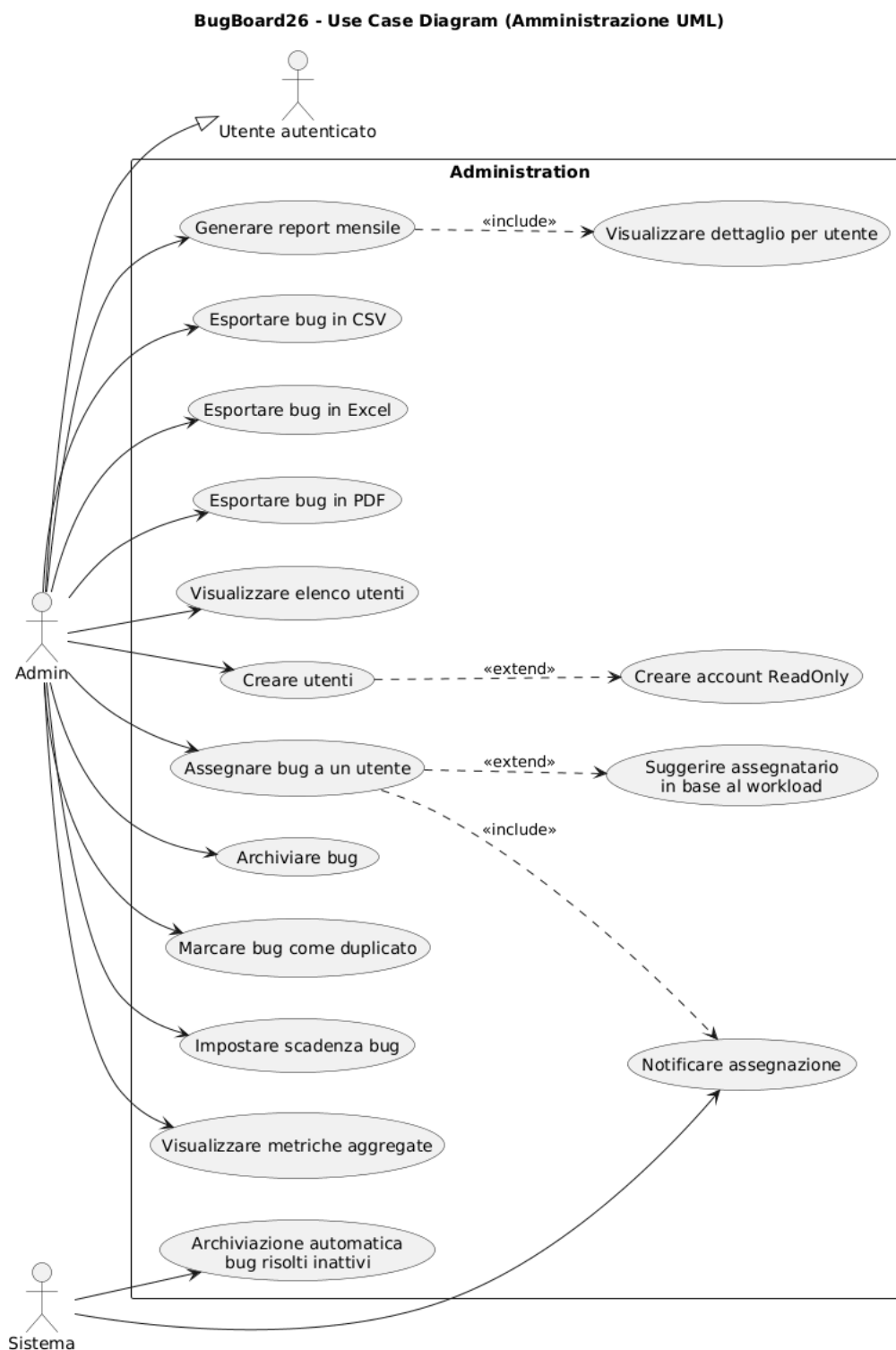


Figura 3: Diagramma dettagliato — Amministrazione e funzioni di sistema

5.3 Convenzioni grafiche

- **Attori** (omino stilizzato o rettangolo): entità esterne che interagiscono con il sistema
- **Casi d'uso** (ellissi): funzionalità offerte dal sistema

- **Linee solide:** relazione di associazione tra attore e caso d'uso
- **Frecce tratteggiate «include»:** il caso d'uso base include sempre il caso d'uso puntato
- **Frecce tratteggiate «extend»:** il caso d'uso puntato estende opzionalmente il caso d'uso base

6 Formalizzazione dei Casi d'Uso (Cockburn)

In questa sezione vengono formalizzati quattro casi d'uso significativi, secondo il formalismo tabellare di Alistair Cockburn. Sono stati esclusi i casi d'uso relativi a registrazione e autenticazione, come indicato nelle istruzioni della traccia.

6.1 Caso d'Uso 1: Creazione nuovo bug

Campo	Descrizione
USE CASE #01	CREAZIONE NUOVO BUG
Goal	L'utente vuole segnalare un nuovo bug nel sistema
Preconditions	L'utente deve essere autenticato (ruolo USER o ADMIN)
Success End Condition	Il bug è creato con successo e registrato con un ID univoco
Failed End Condition	Il bug non viene creato a causa di errori di validazione o problemi di sistema
Primary Actor	Utente registrato (USER)

Flusso	Utente	Sistema
Step 1	Accede alla sezione "Nuovo Bug"	Mostra il form con i campi obbligatori
Step 2	Inserisce il titolo del bug	Valida che il titolo non sia vuoto
Step 3	Seleziona il tipo (Question, Bug, Documentation, Feature)	Carica le categorie disponibili
Step 4	Inserisce la descrizione dettagliata	Fornisce un'area di testo
Step 5	Seleziona la priorità opzionale (Low, Medium, High, Urgent)	Mostra le opzioni con colori identificativi
Step 6	Aggiunge label/tag opzionali	Permette selezione multipla da lista
Step 7	Carica immagini allegati (opzionale)	Valida formato (JPEG, PNG, GIF, WebP) e dimensione (max 5 MB)
Step 8	Se previsto dal ruolo Admin, imposta una deadline opzionale	Mostra il selettore data solo agli utenti amministratori
Step 9	Clicca su "Crea Bug"	Valida tutti i campi obbligatori
Step 10		Salva il bug nel database
Step 11		Registra la creazione nello storico delle modifiche
Step 12		Mostra conferma con l'ID del bug creato

EXTENSIONS	Step	Descrizione
Caricamento immagini fallito	7.a	Mostra errore "Formato non supportato"
	7.b	L'utente può riprovare con file diverso
Validazione campi fallita	9.a	Evidenzia i campi con errori
	9.b	Mostra messaggi di errore per ciascun campo
Deadline non consentita al ruolo corrente	8.a	Il campo non viene mostrato agli utenti non Admin
	8.b	Il sistema ignora eventuali tentativi client-side di impostarla senza autorizzazione

6.2 Caso d'Uso 2: Assegnazione bug a un utente

Campo	Descrizione
USE CASE #02	ASSEGNAZIONE BUG A UN UTENTE
Goal	L'amministratore vuole assegnare o riassegnare un bug a un utente del sistema
Preconditions	L'utente deve essere autenticato con ruolo ADMIN; il bug deve esistere nel sistema
Success End Condition	Il bug risulta assegnato al nuovo utente e la modifica è registrata nello storico
Failed End Condition	L'assegnazione fallisce per utente di destinazione non valido
Primary Actor	Admin

Flusso	Utente (Admin)	Sistema
Step 1	Visualizza i dettagli di un bug	Mostra la pagina di dettaglio con tutte le informazioni
Step 2	Clicca su "Assegna"	Apri il form di assegnazione
Step 3	Visualizza la lista dei membri del team con carico di lavoro corrente	Evidenzia l'utente suggerito con meno bug assegnati
Step 4	Seleziona l'utente dalla lista	Valida che l'utente esista e sia assegnabile
Step 5	Clicca su "Assegna"	Verifica i permessi dell'utente richiedente e la presenza di una selezione valida
Step 6		Aggiorna il campo "assegnato a" nel database
Step 7		Registra la modifica nella tabella history
Step 8		Invia notifica al nuovo assegnatario
Step 9		Mostra messaggio di conferma e reindirizza al dettaglio del bug

EXTENSIONS	Step	Descrizione
Utente non trovato	4.a	Mostra errore "Utente non esistente"
	4.b	L'admin può selezionare un altro membro del team
Nessun utente selezionato	5.a	Mostra errore "Seleziona un utente"
	5.b	L'assegnazione non viene inviata finché non viene scelta una destinazione valida

6.3 Caso d'Uso 3: Aggiornamento stato di un bug assegnato

Campo	Descrizione
USE CASE #03	AGGIORNAMENTO STATO DI UN BUG ASSEGNATO
Goal	L'utente vuole aggiornare lo stato di un bug che può modificare
Preconditions	L'utente deve essere autenticato; deve essere Admin oppure assegnatario del bug
Success End Condition	Il bug viene salvato con il nuovo stato e la modifica viene registrata nello storico
Failed End Condition	L'aggiornamento fallisce per permessi insufficienti, dati non validi o bug inesistente
Primary Actor	Utente registrato (USER) assegnato al bug oppure Admin

Flusso	Utente	Sistema
Step 1	Visualizza i dettagli del bug assegnato	Mostra la pagina di dettaglio con lo stato attuale
Step 2	Clicca su "Modifica"	Verifica che l'utente abbia i permessi per intervenire sul bug
Step 3		Mostra il form di modifica con i campi editabili
Step 4	Aggiorna lo stato del bug (<i>Todo</i> , <i>In Progress</i> , <i>Resolved</i>)	Valida il nuovo valore selezionato
Step 5	Eventualmente modifica titolo, descrizione, priorità o label	Mostra i campi consentiti dal ruolo corrente
Step 6	Clicca su "Salva"	Valida i dati inviati
Step 7		Aggiorna il bug nel database
Step 8		Registra la modifica nella tabella history
Step 9		Se lo stato diventa <i>Resolved</i> , invia una notifica al creatore del bug
Step 10		Mostra il dettaglio aggiornato del bug

EXTENSIONS	Step	Descrizione
Permessi insufficienti	2.a	Nega l'accesso alla modifica se l'utente non è Admin né assegnatario
	2.b	Mantiene il bug in sola lettura
Validazione campi fallita	6.a	Evidenzia i campi obbligatori mancanti o non validi
	6.b	Il salvataggio non viene completato finché gli errori non sono corretti
Bug non trovato	1.a	Mostra un messaggio di errore nel dettaglio
	1.b	Interrompe il flusso di modifica

6.4 Caso d'Uso 4: Consultazione report mensile (Admin)

Campo	Descrizione
USE CASE #04	CONSULTAZIONE REPORT MENSILE
Goal	L'amministratore vuole consultare il report mensile dell'attività del sistema per un mese specifico
Preconditions	L'utente deve essere autenticato con ruolo ADMIN
Success End Condition	Il report del mese selezionato viene mostrato con metriche aggregate e dettaglio per utente
Failed End Condition	La generazione fallisce per assenza di dati nel periodo selezionato o errori di sistema
Primary Actor	Admin

Flusso	Utente (Admin)	Sistema
Step 1	Accede alla sezione "Report Mensili"	Mostra la schermata con il selettore del mese
Step 2	Visualizza il mese corrente preimpostato oppure ne seleziona uno differente	Valida che il mese non sia futuro
Step 3	Conferma la scelta del periodo tramite il campo mese	Richiede al backend i dati aggregati del mese selezionato
Step 4		Raccoglie dati dalle tabelle bugs, users e history
Step 5		Calcola metriche aggregate e medie di risoluzione
Step 6		Mostra card riepilogative e tabella di dettaglio per utente
Step 7	Può scegliere "Esporta report completo"	Reindirizza alla sezione export per il download dei dati

EXTENSIONS	Step	Descrizione
Report non disponibile	3.a	Mostra un messaggio di errore per il mese selezionato
	3.b	L'admin può selezionare un altro periodo
Errore nel caricamento	4.a	Registra l'errore nel log
	4.b	Mostra un messaggio di errore all'amministratore
	4.c	Suggerisce di riprovare o selezionare un altro mese

7 Requisiti Non Funzionali e di Sistema

I requisiti non funzionali definiscono le caratteristiche qualitative che il sistema BugBoard26 deve possedere, indipendentemente dalle funzionalità specifiche implementate.

7.1 Usabilità

- **RNF-U1 — Interfaccia responsiva:** L'interfaccia deve essere fruibile su dispositivi desktop, tablet e mobile senza perdita di funzionalità. Il layout si adatta automaticamente alla larghezza dello schermo.
- **RNF-U2 — Feedback immediato:** Ogni operazione che richiede elaborazione (es. creazione bug, caricamento allegati) deve fornire un feedback visivo all'utente entro 500 ms (es. spinner, messaggio di conferma o di errore).
- **RNF-U3 — Messaggi di errore chiari:** I messaggi di errore devono indicare la causa del problema e suggerire un'azione correttiva, senza esporre dettagli tecnici interni.
- **RNF-U4 — Accessibilità:** L'interfaccia deve rispettare le linee guida WCAG 2.1 livello AA, garantendo contrasti adeguati e compatibilità con tecnologie assistive.

7.2 Prestazioni

- **RNF-P1 — Tempo di risposta API:** Le richieste API standard (lista bug, dettaglio bug, login) devono avere un tempo di risposta inferiore a 500 ms in condizioni normali di carico.
- **RNF-P2 — Paginazione:** Il backend espone la lista bug in forma paginata (default 20 elementi per pagina) per contenere i dataset molto grandi; il frontend può aggregare più pagine quando deve offrire una vista continua all'utente.
- **RNF-P3 — Dimensione allegati:** I file immagine caricati non possono superare 5 MB ciascuno. Formati accettati: JPEG, PNG, GIF, WebP.
- **RNF-P4 — Generazione report:** La generazione di un report mensile deve completarsi entro 10 secondi per dataset fino a 10 000 bug.

7.3 Sicurezza

- **RNF-S1 — Autenticazione:** Tutti gli endpoint applicativi protetti richiedono un token JWT valido; restano pubblici solo login/logout e gli endpoint di supporto allo sviluppo (Swagger/H2). Il token ha una durata configurabile (default: 8 ore).
- **RNF-S2 — Autorizzazione per ruolo:** Le operazioni privilegiate (gestione utenti, report, assegnazione bug) sono accessibili esclusivamente agli utenti con ruolo ADMIN.

- **RNF-S3 — Validazione input:** Tutti i dati in input (lato client e lato server) sono validati prima dell'elaborazione. I file caricati sono verificati per tipo MIME e dimensione.
- **RNF-S4 — Trasmissione sicura:** In produzione, tutte le comunicazioni tra client e server avvengono tramite HTTPS. I cookie di sessione sono configurati con i flag **Secure** e **SameSite**.
- **RNF-S5 — Password:** Le password degli utenti sono memorizzate nel database in forma hash (bcrypt). Non vengono mai restituite nelle risposte API.

7.4 Affidabilità e Disponibilità

- **RNF-A1 — Gestione degli errori:** Il backend restituisce codici HTTP appropriati (400, 401, 403, 404, 500) con messaggi descrittivi. Il frontend mostra notifiche comprensibili per l'utente.
- **RNF-A2 — Persistenza dei dati:** I dati sono persistiti su database relazionale (H2 in sviluppo, sostituibile con PostgreSQL in produzione). Nessuna operazione di scrittura avviene esclusivamente in memoria volatile.
- **RNF-A3 — Storico immutabile:** La tabella *History* non permette cancellazione o modifica dei record; ogni cambiamento genera un nuovo record.

7.5 Manutenibilità e Portabilità

- **RNF-M1 — Containerizzazione:** L'applicazione è distribuita tramite Docker (Dockerfile per frontend e backend; `docker-compose.yml` per l'avvio integrato).
- **RNF-M2 — Separazione frontend/backend:** Frontend e backend sono componenti indipendenti che comunicano esclusivamente tramite API REST. Questa architettura consente di aggiornare o sostituire ciascun componente separatamente.
- **RNF-M3 — Configurabilità:** I parametri critici (URL del database, segreto JWT, porta del server, origine CORS) sono configurabili tramite variabili d'ambiente senza modificare il codice sorgente.
- **RNF-M4 — Documentazione API:** Gli endpoint REST sono documentati tramite SpringDoc/OpenAPI (Swagger UI), accessibile in fase di sviluppo.

7.6 Vincoli di Sistema

- **RNF-VS1 — Tecnologie frontend:** React 18, TypeScript, Vite, Tailwind CSS, Radix UI, React Query, React Router.
- **RNF-VS2 — Tecnologie backend:** Java 17, Spring Boot 3.3, Spring Security, Spring Data JPA, database H2 (sviluppo).
- **RNF-VS3 — Browser supportati:** Le ultime due versioni stabili di Chrome, Firefox, Edge e Safari.
- **RNF-VS4 — Requisiti server:** JDK 17+, Maven 3.8+, Node.js 18+ per la build del frontend.

8 Prototipazione visuale attraverso Mock-up

La prototipazione visuale di BugBoard26 è stata realizzata in **Figma** attraverso un file condiviso di progetto, costruito prima con wireframe a bassa fedeltà e poi raffinato in mock-up ad alta fedeltà con componenti riutilizzabili, palette colori e varianti di stato. Le tavole esportate da Figma sono state usate come riferimento per l'implementazione React del prototipo funzionante e successivamente ricontrollate rispetto ai flussi effettivamente implementati.

8.1 Scelte Progettuali

8.1.1 Layout e Navigazione

- **Navigazione principale persistente:** barra inferiore con accesso rapido a Home, lista bug, notifiche e profilo
- **Sezioni contestuali:** le schermate amministrative e di dettaglio presentano header locale con pulsante di ritorno e azioni disponibili nel contesto corrente
- **Layout responsive:** design *mobile-first* realizzato con Tailwind CSS; le griglie si adattano automaticamente a qualsiasi larghezza schermo

8.1.2 Palette Colori e Badge

- **Priorità:** *Urgent* (rosso), *High* (arancione), *Medium* (giallo), *Low* (verde)
- **Stato:** *Todo* (grigio chiaro), *In Progress* (blu), *Resolved* (verde), *Archived* (grigio scuro)
- **Tipo:** *Bug* (rosso), *Feature* (viola), *Question* (azzurro), *Documentation* (grigio)
- **Azioni primarie:** blu (#007BFF); superfici neutre: grigi Tailwind

8.1.3 Componenti UI riutilizzati

Tutti i componenti sono basati su **Radix UI** e **shadcn/ui**: Button, Card, Select, Dialog, Badge, Table, Toast, DropdownMenu. Questo garantisce coerenza visiva e accessibilità WCAG 2.1 out-of-the-box.

8.2 Mock-up delle interfacce utente

Di seguito sono illustrate le schermate principali del prototipo, raggruppate per funzionalità. Le schermate contrassegnate con **[UC0X]** corrispondono direttamente ai casi d'uso formalizzati nella sezione precedente.

8.2.1 Mock-up 1 — Pagina di Login

BugBoard26
Gestione issue tracking per il tuo team

Email
nome@esempio.com

Password

Accedi

[Registrati](#)

Account demo:
Admin: admin@bugboard.com / admin123
Utente: user@bugboard.com / user123
Readonly: readonly@bugboard.com / readonly123

Figura 4: Pagina di Login: form con email e password, pulsante di accesso

- **Scopo:** autenticazione dell'utente al sistema
- **Elementi:** campo email, campo password, pulsante “Accedi”, link alla pagina informativa del progetto
- **Comportamento:** in caso di credenziali errate viene mostrato un messaggio di errore inline; in caso di successo l'utente viene reindirizzato alla Dashboard

8.2.2 Mock-up 2 — Pagina Informativa Richiesta Account

[← Torna al login](#)

Registrazione

ⓘ La registrazione è gestita dall'amministratore. Questo form è solo dimostrativo.

Nome completo
Mario Rossi

Email
mario@esempio.com

Password

Registrati

Contatta l'amministratore per richiedere un account

Figura 5: Pagina Informativa: indicazioni per l'accesso al sistema e richiesta di un account

- **Scopo:** presentare il progetto e spiegare come ottenere un account creato da un amministratore
- **Elementi:** testo descrittivo, contatti o istruzioni per richiedere l'accesso, link al login

- **Comportamento:** nessuna registrazione self-service; l'utente viene indirizzato verso il canale corretto per la creazione dell'account

8.2.3 Mock-up 3 — Home / Dashboard

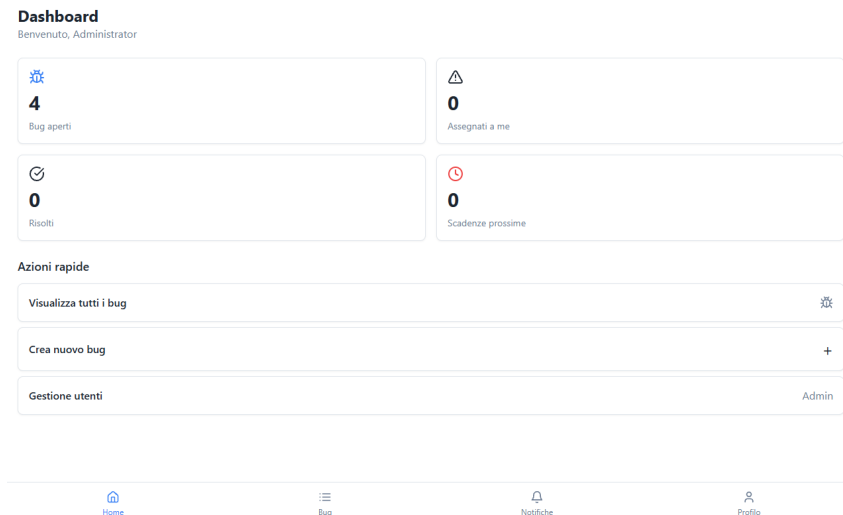


Figura 6: Home Dashboard: panoramica delle metriche principali e accessi rapidi

- **Scopo:** fornire una panoramica immediata dello stato del sistema dopo il login
- **Elementi:** card riepilogative delle metriche principali, saluto iniziale e area “Azioni rapide”
- **Accessi rapidi:** da qui si può navigare alla lista bug, al form di creazione e, per gli Admin, alla gestione utenti

8.2.4 Mock-up 4 — Lista Bug con Filtri

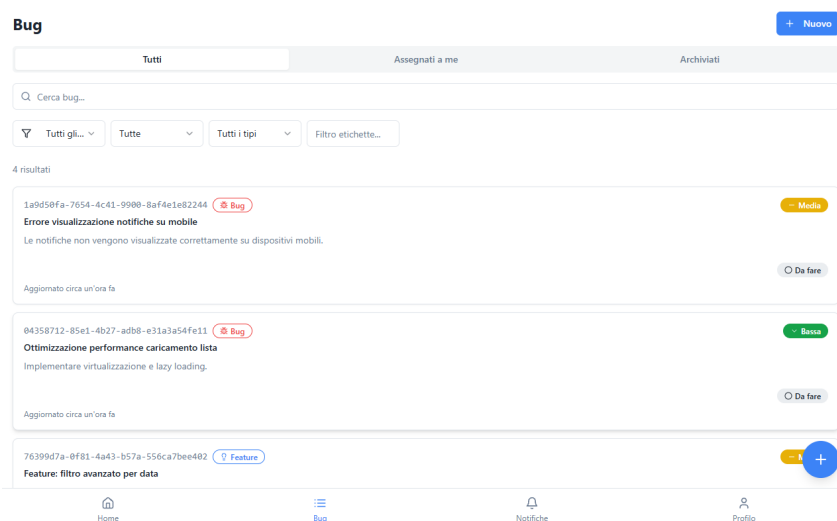


Figura 7: Lista Bug: elenco di card filtrabili per tipo, stato, priorità e ricerca testuale

- **Scopo:** visualizzazione e filtraggio dell'elenco completo dei bug

- **Elementi:** tab “Tutti / Assegnati a me / Archiviati”, barra filtri orizzontale, campo ricerca full-text, card bug con badge colorati
- **Interazione:** il clic su una card apre il dettaglio del bug; i filtri si applicano in tempo reale e il frontend ricompone le pagine restituite dal backend

8.2.5 Mock-up 5 — Dettaglio Bug

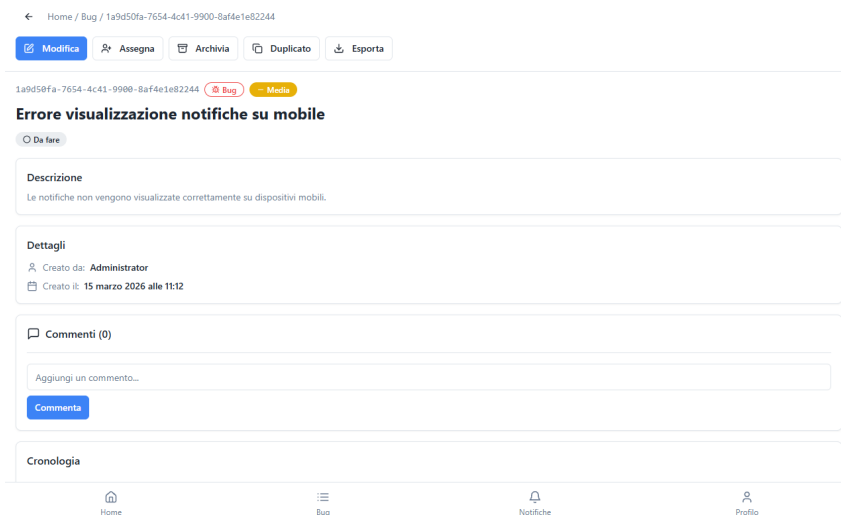


Figura 8: Dettaglio Bug: informazioni complete, commenti, storico modifiche e azioni disponibili

- **Scopo:** visualizzazione completa di un bug specifico e gestione delle azioni disponibili
- **Elementi:** header con breadcrumb locale, titolo, stato e priorità; sezioni descrizione, dettagli, etichette, allegati, commenti e storico; barra azioni orizzontale sticky (modifica, assegna, archivia, duplica, esporta)
- **Azioni visibili:** dipendono dal ruolo dell’utente e dai permessi sul bug corrente

8.2.6 Mock-up 6 — Form Creazione/Modifica Bug [UC01]

Figura 9: Form Creazione Bug (UC01): campi titolo, tipo, priorità opzionale, descrizione, label, allegati

- **Caso d'uso:** UC01 — Creazione nuovo bug
- **Elementi:** campo titolo, selettore tipo, selettore priorità opzionale, area di testo descrizione, selezione label multipla, upload allegati immagine, pulsante “Salva”; il campo deadline compare solo per Admin
- **Validazione:** i campi obbligatori (titolo e descrizione) sono evidenziati se lasciati vuoti; priorità e allegati sono opzionali, mentre formato e dimensione degli allegati sono verificati prima dell'invio

8.2.7 Mock-up 7 — Assegnazione Bug [UC02]

The mock-up shows a screen titled "← Assegna Bug". At the top, there is a text input field containing "Bug: Errore visualizzazione notifiche su mobile". Below this is a suggestion box that says "Suggerimento: Administrator ha il carico di lavoro più basso (0 bug assegnati)". The main section is titled "Seleziona membro del team" and contains a list of two team members. The first member is "Administrator" with email "admin@bugboard.com", marked as "Consigliato" (Recommended), and has "0 bug assegnati" (0 bugs assigned). The second member is "Regular User" with email "user@bugboard.com" and also has "0 bug assegnati". At the bottom, there is a large blue button labeled "Assegna" and a smaller white button labeled "Annulla".

Figura 10: Assegnazione Bug (UC02): selezione dell'utente a cui assegnare il bug

- **Caso d'uso:** UC02 — Assegnazione bug a un utente (solo Admin)
- **Elementi:** select per la scelta dell'utente assegnatario, pulsante conferma
- **Accesso:** questa schermata è accessibile solo agli utenti con ruolo Admin; un USER viene reindirizzato alla Dashboard

8.2.8 Mock-up 8 — Notifiche

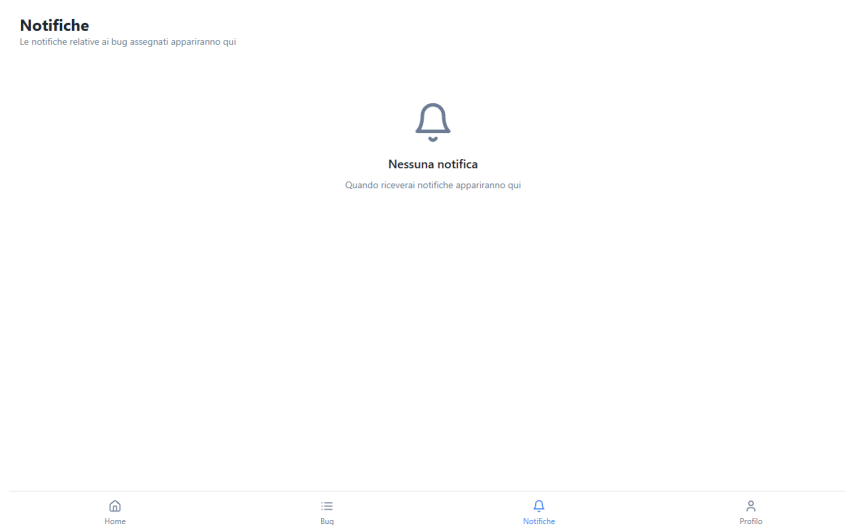


Figura 11: Centro Notifiche: lista degli aggiornamenti sui bug seguiti dall'utente

- **Scopo:** mostrare all'utente gli aggiornamenti recenti sui bug che lo riguardano (assegnazioni, cambi stato, nuovi commenti)
- **Elementi:** lista notifiche con timestamp, tipo evento e link al bug correlato; indicatore notifiche non lette

8.2.9 Mock-up 9 — Profilo Utente

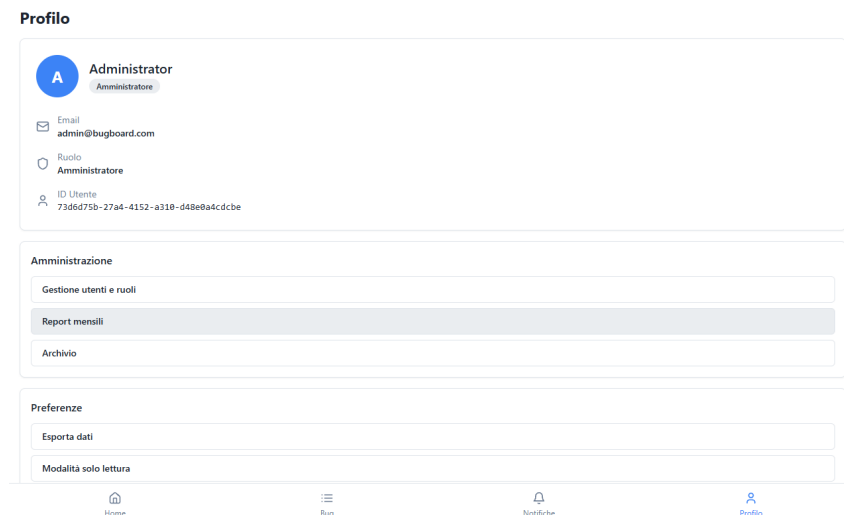


Figura 12: Profilo Utente: consultazione dati account e accesso alle sezioni personali

- **Scopo:** consentire all'utente di consultare i dati del proprio account e raggiungere rapidamente le funzioni personali disponibili
- **Elementi:** nome, email, ruolo, ID utente, scorciatoie a export e modalità sola lettura; per gli Admin compaiono anche collegamenti a gestione utenti, report e archivio

8.2.10 Mock-up 10 — Report Mensile [UC04]

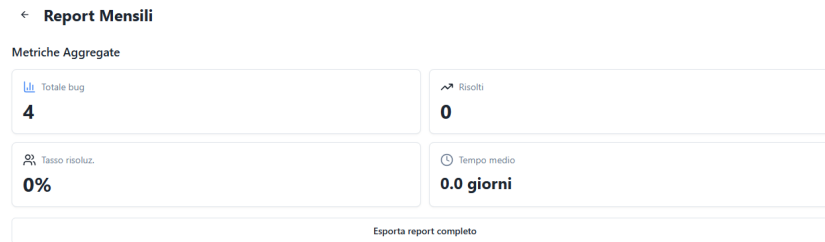


Figura 13: Report Mensile Admin (UC04): consultazione delle statistiche aggregate del mese e dettaglio per utente

- **Caso d'uso:** UC04 — Consultazione report mensile (solo Admin)
- **Elementi:** selettore mese, statistiche aggregate, tabella riassuntiva per utente e collegamento alla sezione export
- **Accesso:** solo Admin; gli altri utenti vengono reindirizzati alla Dashboard

8.2.11 Mock-up 11 — Gestione Utenti (Admin)

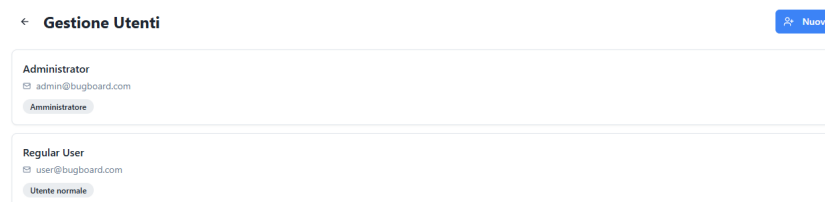


Figura 14: Gestione Utenti Admin: lista utenti con ruoli e form di creazione account

- **Scopo:** permettere all'Admin di gestire gli account del sistema tramite visualizzazione utenti e creazione di nuovi account con ruolo
- **Elementi:** tabella utenti con email, nome e ruolo; form per creazione di nuovi account Admin, User o ReadOnly

8.2.12 Mock-up 12 — Archivio Bug

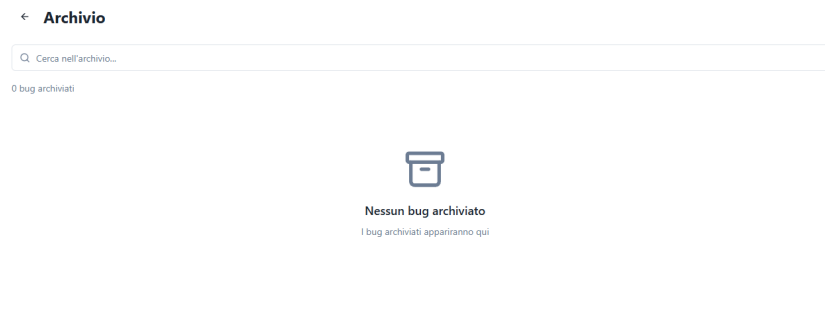


Figura 15: Archivio Bug: lista dei bug archiviati con possibilità di consultazione

- **Scopo:** visualizzare i bug archiviati per storico e consultazione senza influire sulla lista attiva
- **Elementi:** elenco dei bug in stato *Archived* con ricerca; solo Admin può accedere alla sezione e archiviare manualmente i bug dalla vista di dettaglio

8.2.13 Mock-up 13 — Export Dati

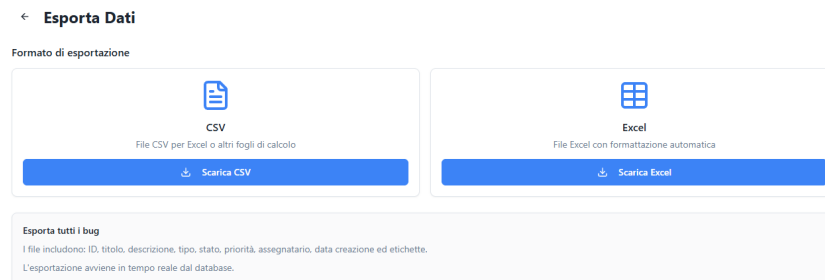


Figura 16: Export Dati: schermata di esportazione con opzioni CSV ed Excel; il PDF è supportato nella versione implementata

- **Scopo:** consentire l'esportazione dei dati per analisi esterne
- **Elementi:** pulsanti “Scarica CSV” e “Scarica Excel” visibili nel mock-up; nella versione implementata la stessa pagina espone anche il download PDF

8.3 Osservazioni emerse dalla prototipazione

La costruzione dei mock-up in Figma e la loro successiva validazione sul prototipo funzionante hanno permesso di identificare alcune scelte migliorative rispetto all'analisi iniziale:

- **Chiarezza dei ruoli:** è stato necessario rendere esplicito nel UI quali azioni sono riservate all'Admin (assegnazione, report, gestione utenti), aggiungendo reindirizzamenti automatici per utenti non autorizzati
- **Filtri immediati:** la lista bug con filtri in tempo reale si è rivelata la funzionalità più usata nelle sessioni di test, confermando la scelta di renderla la pagina principale
- **Badge colorati:** l'uso di badge cromatici per stato, priorità e tipo ha migliorato significativamente la leggibilità della lista bug senza aprire il dettaglio
- **Navigazione mobile:** l'adozione di una bottom navigation persistente ha reso più immediato l'accesso alle sezioni principali sui dispositivi mobili

9 Design del Sistema

9.1 Descrizione dell'architettura

BugBoard26 adotta un'architettura **client-server a tre livelli** (Three-Tier Architecture), separando nettamente le responsabilità tra *presentazione*, *logica di business* e *persistenza dei dati*.

Livello	Descrizione
Presentazione (Frontend)	Single Page Application React che comunica con il backend tramite API REST.
Logica di Business (Backend)	Applicazione Spring Boot che espone endpoint RESTful, gestisce autenticazione JWT e implementa le regole di business.
Persistenza (Database)	Database relazionale (H2 in sviluppo, PostgreSQL in produzione) gestito tramite JPA/Hibernate.

9.1.1 Deployment Diagram

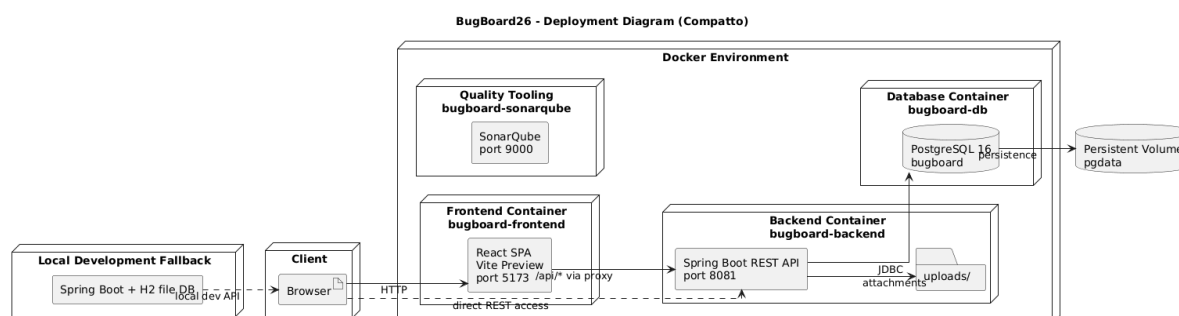


Figura 17: Deployment Diagram — BugBoard26

Il deployment diagram mostra la distribuzione fisica dei componenti tra i container Docker e l'ambiente di sviluppo locale.

9.1.2 Component Diagram

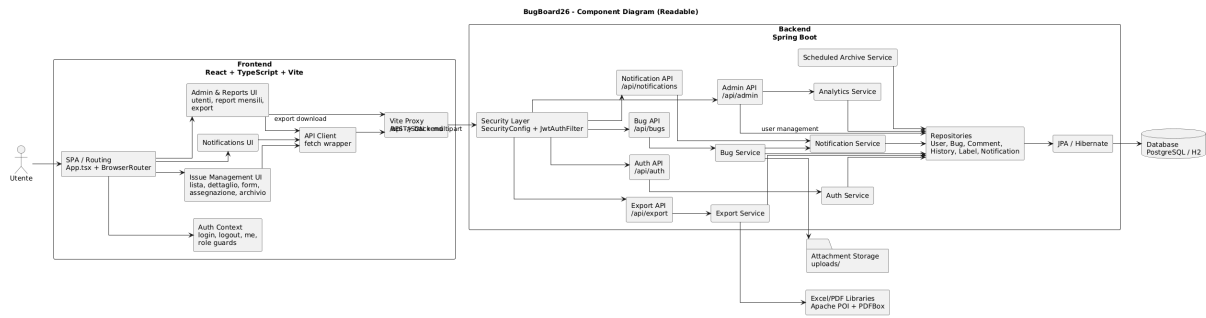


Figura 18: Component Diagram — BugBoard26

Il component diagram illustra i componenti software del sistema e le loro dipendenze, organizzati per layer architetturale.

9.1.3 Diagrammi di approfondimento architetturale

Accanto ai diagrammi sintetici principali, sono stati inclusi anche diagrammi UML più dettagliati, utili per descrivere con maggior precisione i layer interni del backend e le dipendenze effettivamente presenti nel repository.

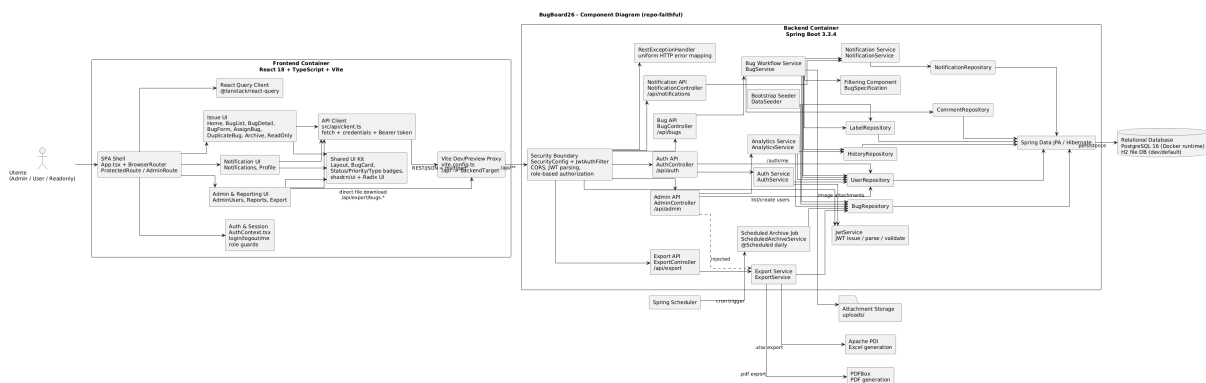


Figura 19: Component Diagram dettagliato, allineato alla struttura effettiva del repository

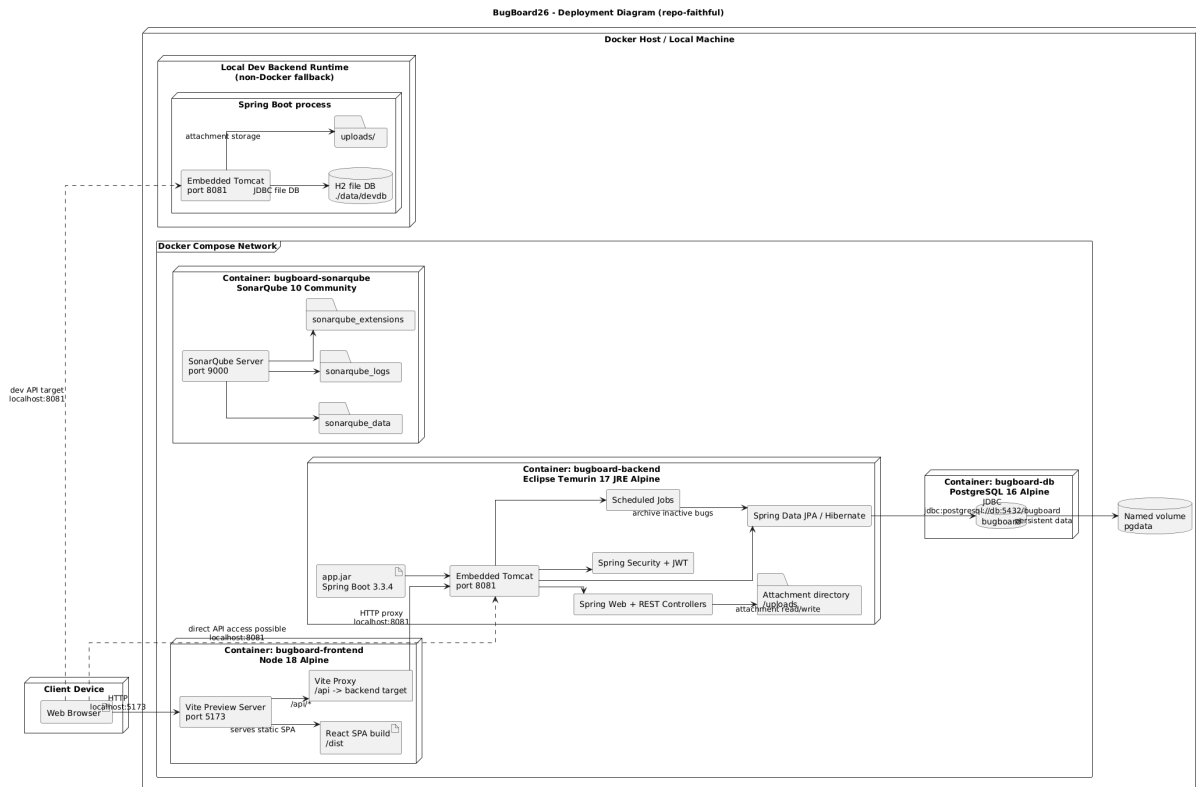


Figura 20: Deployment Diagram dettagliato con ambiente Docker, fallback locale e volumi persistenti

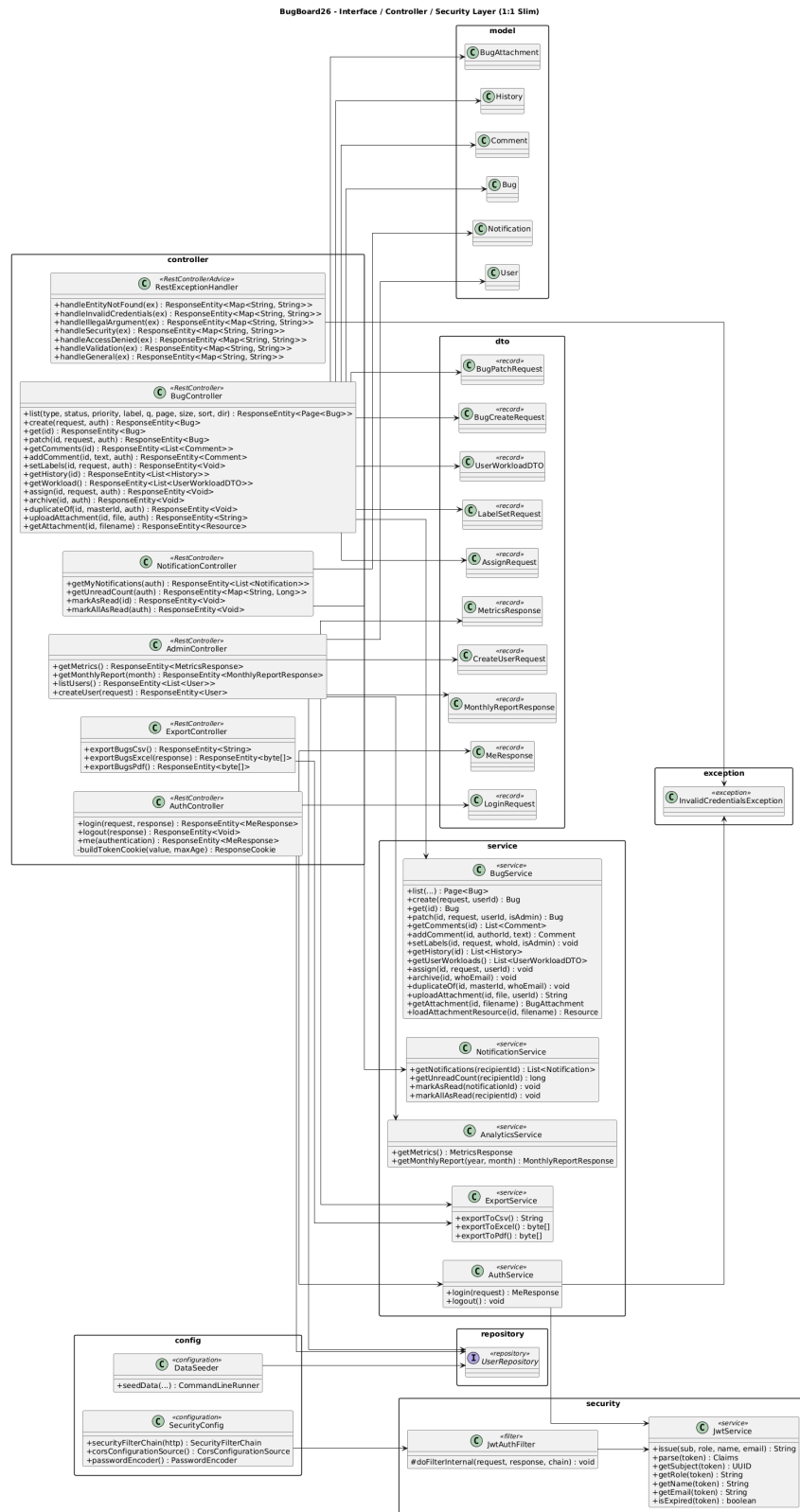


Figura 21: Vista detallada del layer Interface / Controller / Security

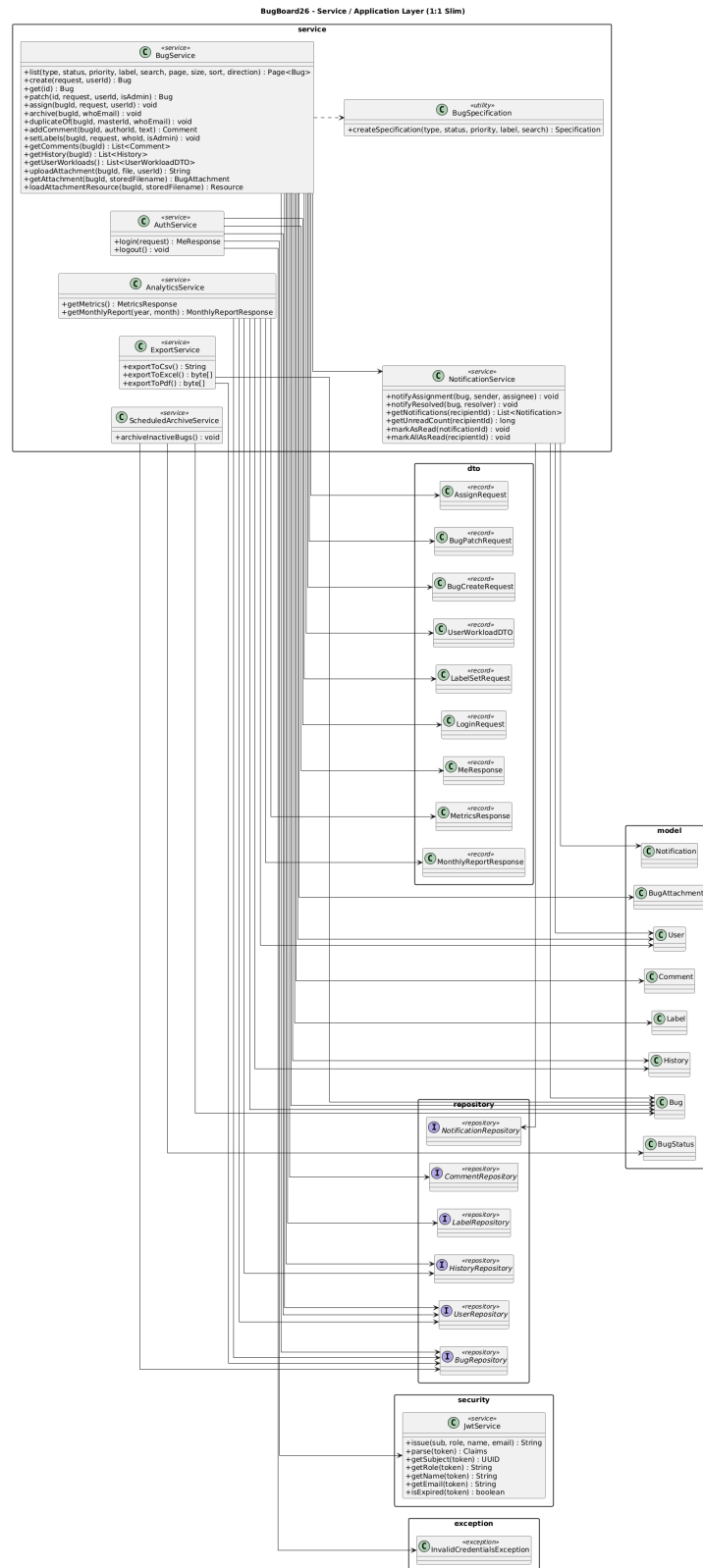


Figura 22: Vista dettagliata del layer Service / Application

9.2 Criteri di design adottati

Separazione delle responsabilità (*Separation of Concerns*)

Ogni livello ha una responsabilità ben definita: il frontend si occupa esclusivamente della presentazione e dell'interazione utente, il backend della logica di business e dell'autoriz-

zazione, il database della persistenza. Questa separazione favorisce la manutenibilità e la testabilità.

Stateless Authentication

L'autenticazione è gestita tramite token JWT, eliminando la necessità di sessioni server-side. Ogni richiesta contiene il token necessario per l'identificazione, facilitando la scalabilità orizzontale.

API-First Design

Il backend espone un'API RESTful documentata tramite OpenAPI/Swagger (SpringDoc). Questo approccio consente lo sviluppo indipendente di frontend e backend, facilita l'integrazione con altri client e permette il testing automatico delle API.

Convention over Configuration

L'utilizzo di Spring Boot riduce drasticamente la configurazione boilerplate grazie alle *auto-configurations* e alle convenzioni del framework.

Containerizzazione

L'intero stack è containerizzabile tramite Docker e orchestrabile con Docker Compose, garantendo riproducibilità dell'ambiente e facilità di deployment.

9.3 Scelte tecnologiche

9.3.1 Frontend

Tecnologia	Versione	Motivazione
React	18.x	Libreria UI component-based matura e con vasto ecosistema; facilita la creazione di SPA reattive.
TypeScript	5.x	Tipizzazione statica che riduce i bug a runtime e migliora la manutenibilità del codice.
Vite	5.4.x	Build tool moderno, significativamente più veloce di Webpack grazie a ESBuild e HMR nativo.
Tailwind CSS	3.x	Framework CSS utility-first che accelera lo sviluppo della UI e garantisce consistenza visiva.
Radix UI	–	Componenti accessibili e non stilizzati (<i>headless</i>), conformi a WAI-ARIA, personalizzabili con Tailwind.
React Query	5.x	Infrastruttura condivisa per la gestione dello stato server-side; nell'implementazione corrente è predisposto tramite <code>QueryClientProvider</code> e integrato gradualmente accanto alle chiamate API manuali.
React Router	6.30.x	Routing dichiarativo per SPA con supporto a route protette e navigazione client-side.

9.3.2 Backend

Tecnologia	Versione	Motivazione
Java	17	Linguaggio robusto e type-safe, con record per DTO concisi e ottimo supporto nell'ecosistema Spring Boot.
Spring Boot	3.3.4	Framework enterprise standard per applicazioni Java; auto-configuration, starter dependencies, embedded server.
Spring Security	–	Gestione completa di autenticazione e autorizzazione; integrazione nativa con JWT.
Spring Data JPA	–	Astrazione ORM che elimina il codice boilerplate per l'accesso ai dati; supporto a Specification per query dinamiche.
H2 Database	–	Database embedded per lo sviluppo locale; nessuna installazione richiesta, avvio immediato.
PostgreSQL	16	Database relazionale per produzione; robusto, conforme ACID, supporto nativo a UUID.
JWT	0.12.5	Libreria per la creazione e validazione di token JWT; API fluente e sicura.
SpringDoc	2.6.0	Generazione automatica della documentazione OpenAPI 3.0 dalle annotazioni Spring.
Apache POI	5.3.0	Generazione di file Excel (.xlsx) per l'export dei dati bug.
PDFBox	2.0.30	Generazione di documenti PDF per l'export dell'elenco bug.

9.3.3 Testing

Tecnologia	Motivazione
JUnit 5	Framework di testing standard per Java; supporto a test parametrizzati, display names, extensions.
Mockito 5	Libreria di mocking che consente l'isolamento delle dipendenze nei test di unità.
AssertJ	Asserzioni fluenti che migliorano la leggibilità dei test.

9.3.4 DevOps e Deployment

Tecnologia	Motivazione
Docker	Containerizzazione per ambienti riproducibili; Dockerfile multi-stage per build ottimizzati.
Docker Compose	Orchestrazione dei servizi del sistema (database, backend, frontend e tooling di qualità) con un solo comando.
Git / GitHub	Version control distribuito; collaborazione, code review, CI/CD integration.
Maven	Build automation per il backend; gestione dipendenze, lifecycle standardizzato.
npm	Package manager per il frontend; gestione dipendenze e script di build.

9.4 Comunicazione tra i livelli

La comunicazione tra frontend e backend avviene tramite API REST su HTTP/HTTPS:

- **Formato dati:** JSON (Content-Type: `application/json`)
- **Autenticazione:** Token JWT trasportato tramite cookie `HttpOnly (token)` oppure header `Authorization: Bearer <token>`
- **Proxy in sviluppo:** Vite espone il frontend su `localhost:8080` e reindirige `/api/*` al backend configurato (default `localhost:18081`), evitando problemi CORS durante lo sviluppo
- **Endpoint principali:**
 - POST `/api/auth/login` — Autenticazione
 - GET `/api/auth/me` — Profilo dell'utente autenticato
 - GET/POST `/api/bugs` — Listing e creazione bug
 - PATCH `/api/bugs/{id}` — Modifica bug
 - POST `/api/bugs/{id}/assign` — Assegnazione (admin)
 - GET `/api/notifications` — Centro notifiche
 - GET `/api/bugs/workload` — Carico di lavoro utenti per suggerimento assegnatario
 - GET `/api/admin/metrics` — Metriche (admin)
 - GET `/api/export/bugs.csv` — Export CSV
 - GET `/api/export/bugs.xlsx` — Export Excel
 - GET `/api/export/bugs.pdf` — Export PDF

9.5 Sicurezza

- **Password:** hash BCrypt con salt automatico
- **CSRF:** disabilitato (architettura stateless con JWT)
- **CORS:** configurazione restrittiva con whitelist degli origin consentiti
- **Sessioni:** nessuna sessione server-side (`SessionCreationPolicy.STATELESS`)
- **Autorizzazione:** controllo a livello di servizio basato sul ruolo dell'utente (ADMIN, USER, READONLY)
- **Cookie JWT:** attributi `HttpOnly` e `SameSite=Lax` per mitigare XSS e CSRF

10 Design del Software

10.1 Schema per la persistenza dati

Il database relazionale è gestito tramite JPA/Hibernate con strategia `ddl-auto=update`. Di seguito lo schema entità-relazione con tutte le tabelle, colonne e vincoli.

10.1.1 Tabella users

Colonna	Tipo	Vincoli	Note
id	UUID	PK, AUTO	Identificativo univoco
email	VARCHAR	UNIQUE, NOT NULL	Indirizzo email
password_hash	CHAR(60)	NOT NULL	Hash BCrypt
name	VARCHAR	NOT NULL	Nome visualizzato
role	VARCHAR	NOT NULL	Enum: ADMIN, USER, READ-ONLY
created_at	TIMESTAMP		Data di registrazione

10.1.2 Tabella bugs

Colonna	Tipo	Vincoli	Note
id	UUID	PK, AUTO	Identificativo univoco
title	VARCHAR	NOT NULL	Titolo del bug
description	TEXT	NOT NULL	Descrizione dettagliata
type	VARCHAR	NOT NULL	Enum: QUESTION, BUG, DOCUMENTATION, FEATURE
status	VARCHAR		Enum: TODO, IN_PROGRESS, RESOLVED, ARCHIVED
priority	VARCHAR		Enum: LOW, MEDIUM, HIGH, URGENT
archived	BOOLEAN	default false	Flag di archiviazione
created_at	TIMESTAMP		Data di creazione
updated_at	TIMESTAMP		Data ultimo aggiornamento
deadline	TIMESTAMP	nullable	Scadenza opzionale
duplicate_of	UUID	FK → bugs.id	Bug duplicato (self-ref)
created_by	UUID	FK → users.id	Creatore
assignee_id	UUID	FK → users.id	Assegnatario

10.1.3 Tabella comments

Colonna	Tipo	Vincoli	Note
id	UUID	PK, AUTO	Identificativo univoco
text	TEXT	NOT NULL	Testo del commento
bug_id	UUID	FK → bugs.id, NOT NULL	Bug associato
author_id	UUID	FK → users.id, NOT NULL	Autore
created_at	TIMESTAMP		Data di creazione

10.1.4 Tabella history

Colonna	Tipo	Vincoli	Note
id	UUID	PK, AUTO	Identificativo univoco
bug_id	UUID	FK → bugs.id, NOT NULL	Bug associato
who_id	UUID	FK → users.id, NOT NULL	Utente esecutore
action	VARCHAR	NOT NULL	Enum: CREATE, UPDATE, ASSIGN, COMMENT, ARCHIVE, DUPLICATE, LABELS_SET, ATTACHMENT_UPLOADED
details	TEXT	nullable	Dettagli del cambiamento
created_at	TIMESTAMP		Timestamp dell'azione

10.1.5 Tabella labels

Colonna	Tipo	Vincoli	Note
id	UUID	PK, AUTO	Identificativo univoco
name	VARCHAR	UNIQUE, NOT NULL	Nome della label
created_at	TIMESTAMP		Data di creazione

10.1.6 Tabella di join bug_labels

Colonna	Tipo	Vincoli	Note
bug_id	UUID	FK → bugs.id	Relazione multi-a-molti Bug ↔ Label
label_id	UUID	FK → labels.id	

10.1.7 Tabella notifications

Colonna	Tipo	Vincoli	Note
id	UUID	PK, AUTO	Identificativo univoco
recipient_id	UUID	FK → users.id, NOT NULL	Destinatario della notifica
sender_id	UUID	FK → users.id	Utente che ha generato l'evento, se presente
bug_id	UUID	FK → bugs.id, NOT NULL	Bug correlato alla notifica
message	TEXT	NOT NULL	Messaggio mostrato nel centro notifiche
is_read	BOOLEAN	default false	Stato di lettura
created_at	TIMESTAMP		Data di creazione

10.1.8 Collection table bug_attachments

Colonna	Tipo	Vincoli	Note
bug_id	UUID	FK → bugs.id, NOT NULL	Bug proprietario dell'allegato
stored_filename	VARCHAR	NOT NULL	Nome interno del file memorizzato
original_filename	VARCHAR	NOT NULL	Nome originale del file caricato
content_type	VARCHAR	NOT NULL	MIME type dell'allegato
file_size	BIGINT	NOT NULL	Dimensione in byte
uploaded_at	TIMESTAMP	NOT NULL	Data di caricamento

10.1.9 Diagramma E-R semplificato

Il diagramma E-R semplificato aggiornato, coerente con l'implementazione corrente del backend, è riportato in Figura 23.

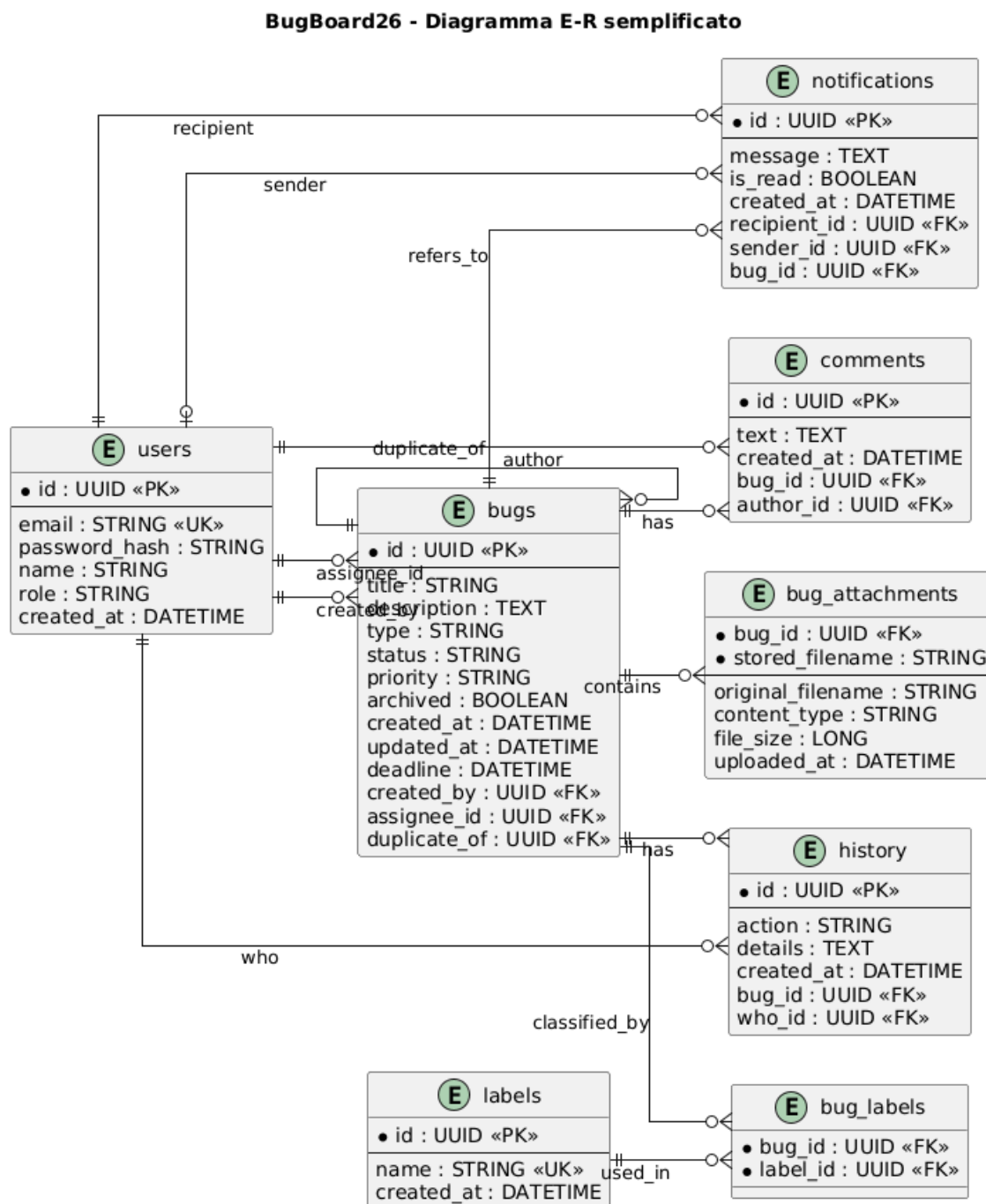


Figura 23: Diagramma E-R semplificato del database di BugBoard26

10.1.10 Diagramma di persistenza dettagliato

Per affiancare la vista E-R semplificata, in Figura 24 è riportata anche una rappresentazione UML più dettagliata del layer di persistenza e dominio, utile a evidenziare repository, entità, enumerazioni e associazioni principali in modo più vicino al codice.

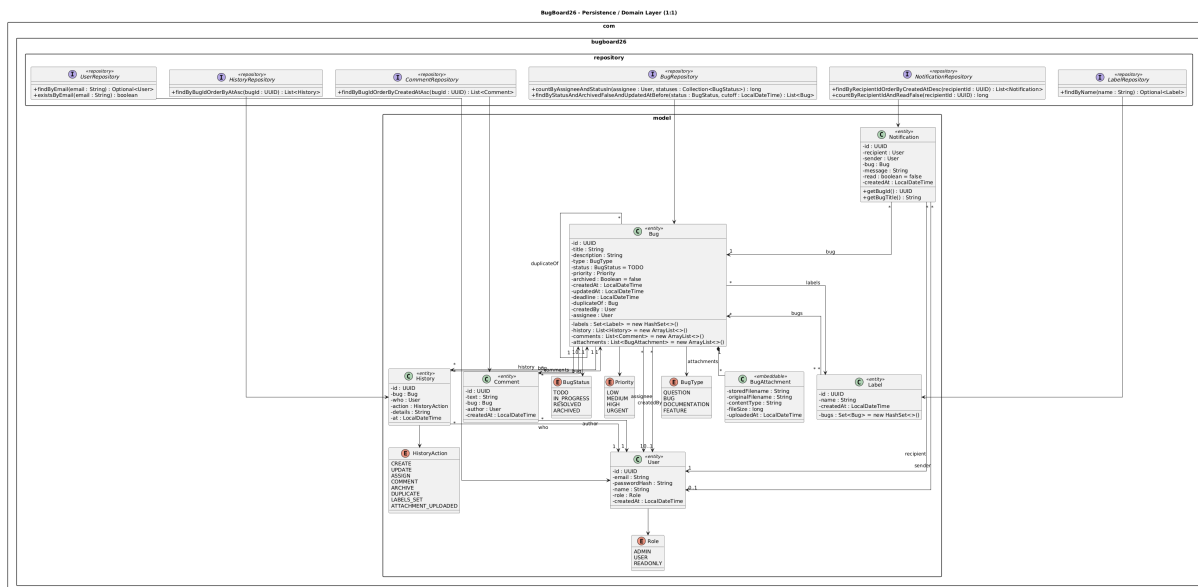


Figura 24: Diagramma UML dettagliato del layer di persistenza e dominio

10.2 Diagramma delle classi di design

I diagrammi UML di analisi e design sono stati modellati tramite un **CASE tool** dedicato, nello specifico **StarUML**, e successivamente esportati e rifiniti per l'inclusione nella documentazione finale. Il diagramma delle classi di design è organizzato secondo il pattern architetturale **layered architecture** con i seguenti package:

model

Entità JPA che mappano le tabelle del database: User, Bug, Comment, History, Label, Notification, BugAttachment e le enumerazioni Role, BugType, BugStatus, Priority, HistoryAction.

repository

Interfacce Spring Data JPA che estendono JpaRepository<T, UUID>: UserRepository, BugRepository (+ JpaSpecificationExecutor), CommentRepository, LabelRepository, HistoryRepository, NotificationRepository.

service

Classi di business logic: BugService, AuthService, AnalyticsService, ExportService, NotificationService, ScheduledArchiveService, oltre a BugSpecification per la costruzione delle query dinamiche.

dto Record Java per i dati di trasferimento: BugCreateRequest, BugPatchRequest, AssignRequest, LabelSetRequest, LoginRequest, CreateUserRequest, MeResponse, MetricsResponse, MonthlyReportResponse, MonthlyUserReportResponse, UserWorkloadDTO.

controller

Controller REST: BugController (/api/bugs), AuthController (/api/auth), AdminController (/api/admin), NotificationController (/api/notifications), ExportController (/api/export) e RestExceptionHandler per la gestione centralizzata degli errori HTTP.

security

Infrastruttura di sicurezza: JwtService (generazione/validazione token), JwtAuthFilter (filtro per estrarre e verificare il JWT).

config

Configurazione dell'applicazione: **SecurityConfig** (CORS, CSRF, filter chain), **DataSeeder** (dati iniziali per lo sviluppo).

exception

Eccezioni applicative dedicate, ad esempio **InvalidCredentialsException**, usate per segnalare errori di dominio in modo esplicito.

Il diagramma delle classi di design aggiornato in coerenza con l'implementazione corrente è riportato in Figura 25.

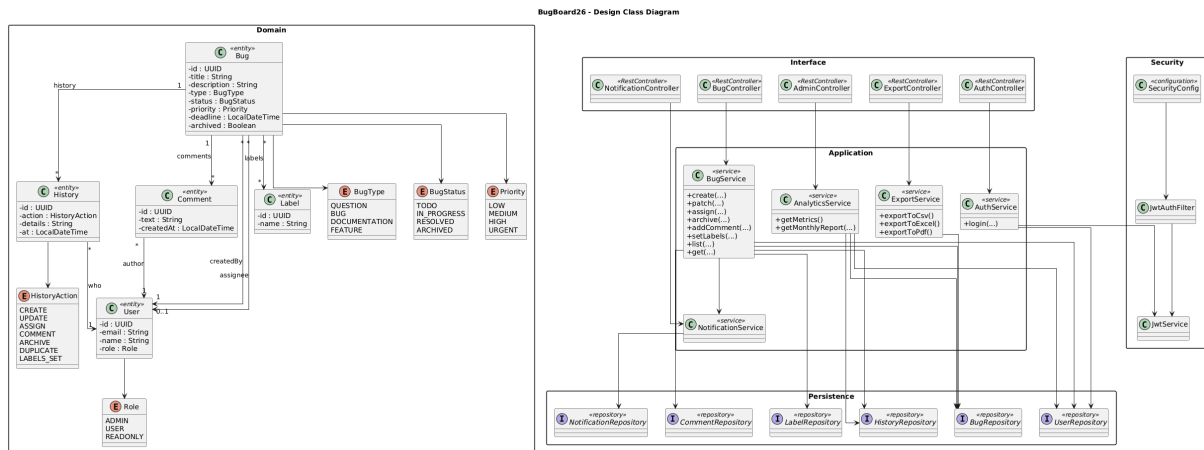


Figura 25: Diagramma delle classi di design — BugBoard26

10.3 Diagramma comportamentale — Sequence Diagram

Il sequence diagram in Figura 26 illustra il flusso completo dell'operazione di **creazione di un nuovo bug**, dalla compilazione del form nell'interfaccia utente fino alla persistenza nel database. Questo caso d'uso è stato scelto in quanto coinvolge tutti i layer architetturali e presenta logica condizionale non banale (gestione labels, validazione utente).

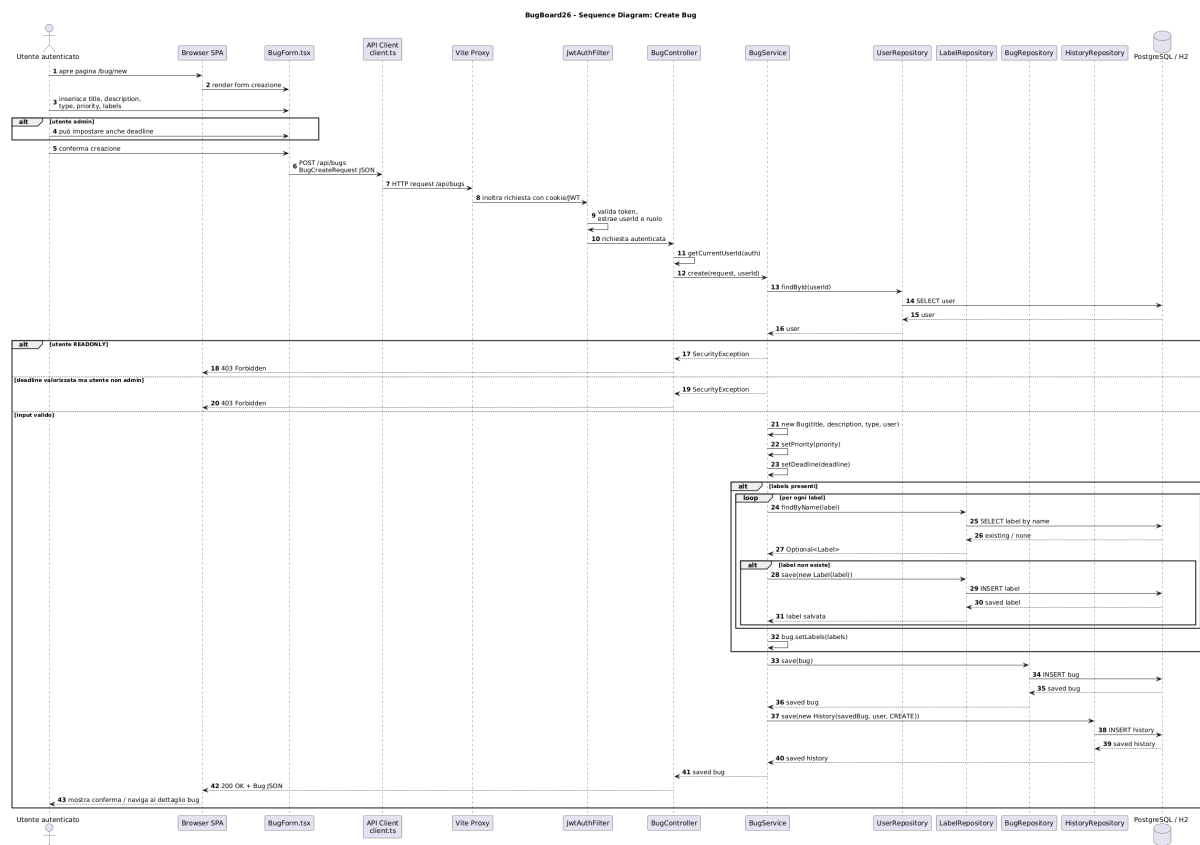


Figura 26: Sequence Diagram — Creazione di un bug

Per completezza è stato incluso anche un secondo sequence diagram relativo all'assegnazione di un bug, con evidenza esplicita del controllo ruolo amministratore, della scelta dell'assegnatario e della generazione della notifica.

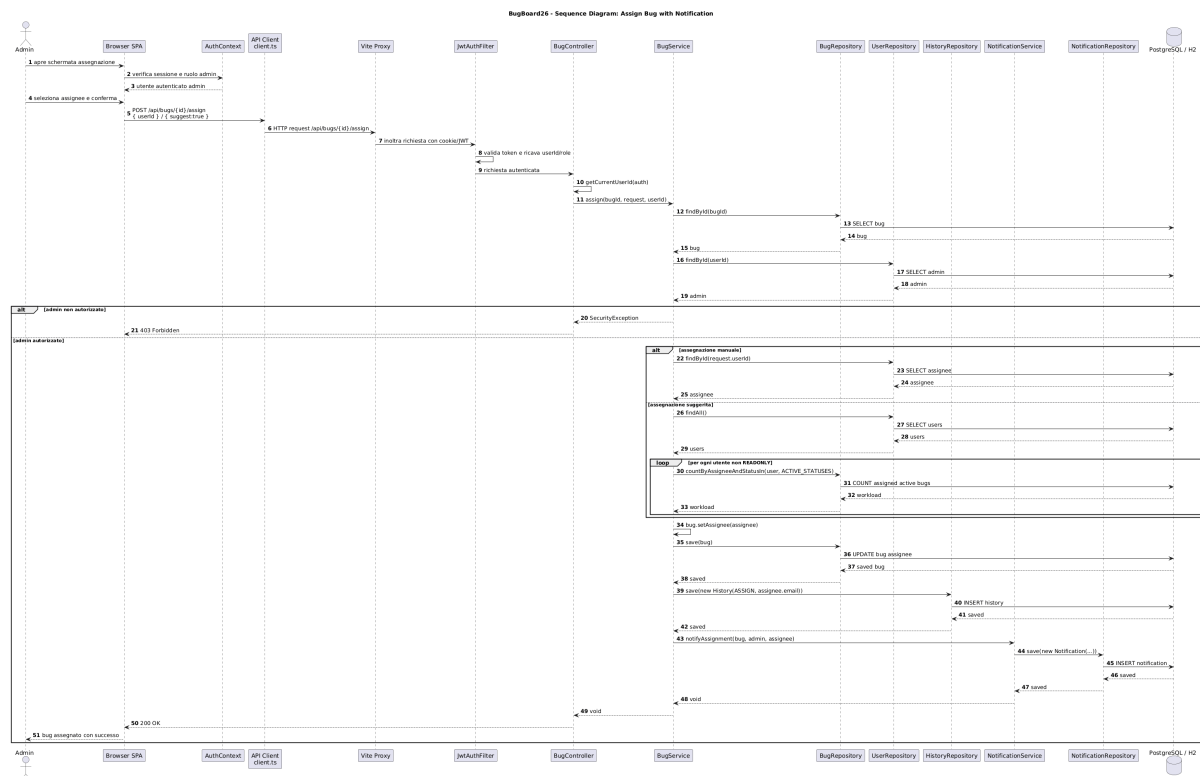


Figura 27: Sequence Diagram — Assegnazione di un bug con notifica

10.4 Scelte di software design

10.4.1 Pattern Repository

Ogni entità dispone di un'interfaccia repository dedicata che estende `JpaRepository`. Questo pattern separa la logica di accesso ai dati dalla logica di business, favorendo la testabilità tramite mock objects.

`BugRepository` estende anche `JpaSpecificationExecutor`, consentendo la costruzione di query dinamiche (filtri per tipo, stato, priorità, label, ricerca testuale) senza codice SQL custom.

10.4.2 Pattern Service Layer

La logica di business è concentrata nei servizi (`BugService`, `AuthService`), che orchestrano le operazioni tra più repository. I controller sono *thin*: delegano immediatamente ai servizi senza contenere logica di business.

10.4.3 Record Java per i DTO

I DTO sono implementati come **record** Java, che garantiscono immutabilità, auto-generazione di `equals/hashCode/toString`, e riduzione del codice boilerplate. Le annotazioni di validazione (`@NotBlank`, `@NotNull`, `@NotEmpty`) sono applicate direttamente sui componenti del record.

10.4.4 Specification Pattern per le query

La classe `BugSpecification` implementa il pattern Specification di Spring Data, componendo predicati JPA Criteria in modo dichiarativo. I filtri (tipo, stato, priorità, label, ricerca full-text su titolo e descrizione) possono essere combinati liberamente dal frontend.

10.4.5 Audit Trail tramite History

Ogni operazione significativa (creazione, modifica, assegnazione, commento, archiviazione) genera automaticamente una entry nella tabella **history**, con riferimento all'utente, all'azione e ai dettagli della modifica. Questo garantisce tracciabilità completa senza librerie esterne.

10.4.6 Autenticazione stateless con JWT

JwtService genera token firmati HMAC-SHA256 con claims personalizzati (ruolo, nome, email). **JwtAuthFilter** intercetta ogni richiesta, estrae il token da cookie o header, e popola il **SecurityContext** di Spring Security. Questo approccio elimina la necessità di sessioni server-side e facilita la scalabilità orizzontale.

10.4.7 Gestione delle autorizzazioni

Il controllo degli accessi è implementato a due livelli:

- **Livello HTTP**: Spring Security filtra le richieste in base all'autenticazione (JWT valido).
- **Livello Service**: le regole di business (solo admin può assegnare; solo assegnatario o admin può cambiare stato) sono verificate nel servizio, lanciando **SecurityException** in caso di violazione.

10.5 Evidenza dell'uso di strumenti di versioning

Il codice sorgente del progetto è ospitato su un repository **GitHub** accessibile ai membri del team di sviluppo.

- **Repository**: <https://github.com/alessandroilprogrammatore/BugBoard26>
- **Release**: la release "Specifica dei requisiti" è stata pubblicata il 19 ottobre 2025
- **Branching strategy**: sviluppo su branch feature, merge su **main** tramite pull request
- **Commit**: messaggi descrittivi che seguono il formato convenzionale (**feat:**, **fix:**, **docs:**, **test:**)
- **Statistiche repository**: 86 commit complessivi e 3 contributor rilevati nella history Git
- **Contributors**: Alessandro Minopoli, Simoalvin (Simone Mollo), PorfirioTramontana
- **Frequenza dei commit**: attività distribuita tra ottobre 2025 e marzo 2026, con una chiara intensificazione nella fase finale del progetto

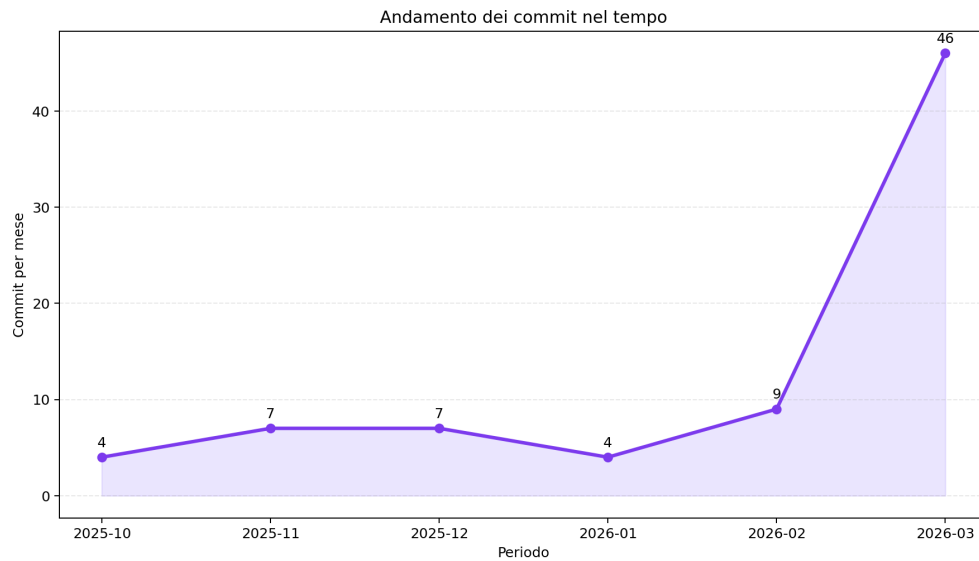


Figura 28: Andamento temporale dei commit nel repository

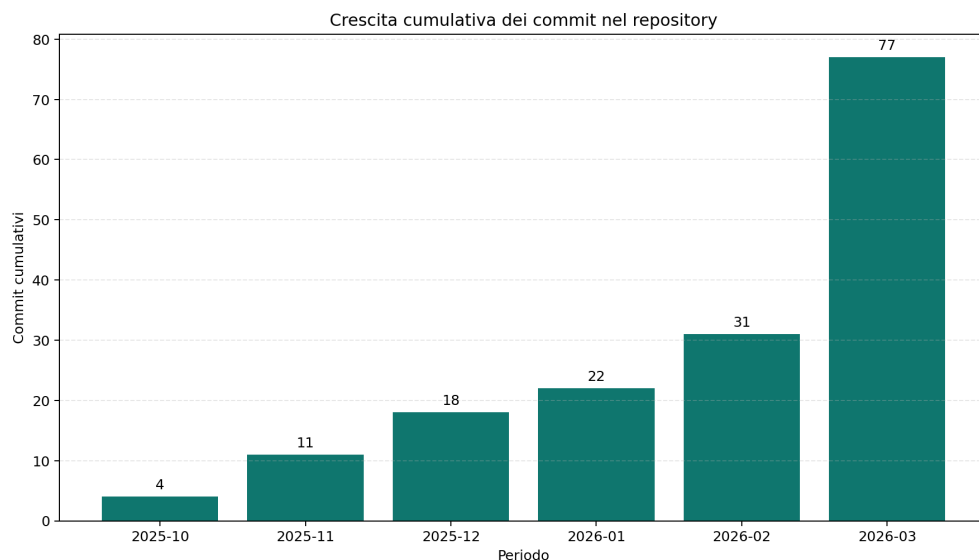


Figura 29: Crescita cumulativa dei commit nel repository

11 Test Plan – Gestione Bug

11.1 Obiettivo

Il test plan descrive le attività di verifica previste per la funzionalità **Gestione Bug** del sistema BugBoard26. L'obiettivo è validare che le operazioni di creazione, assegnazione e modifica di un bug rispettino i requisiti funzionali e le regole di autorizzazione definite nella specifica.

11.2 Funzionalità sotto test

Le funzionalità coperte dal piano sono quelle esposte dal servizio **BugService**:

1. **Creazione di un bug** – metodo `create(BugCreateRequest, UUID)`
2. **Assegnazione di un bug** – metodo `assign(UUID, AssignRequest, UUID)`

3. Modifica di un bug – metodo `patch(UUID, BugPatchRequest, UUID, boolean)`

11.3 Approccio di testing

- **Livello:** test di unità (*unit test*).
- **Tecnica:** *black-box* (classi di equivalenza, valori limite) combinata con ispezione *white-box* (copertura dei rami decisionali interni).
- **Framework:** JUnit 5 + Mockito 5 (via `spring-boot-starter-test`).
- **Isolamento:** le dipendenze esterne (*repository* JPA e servizi ausiliari come `NotificationService`) sono sostituite da *mock objects* iniettati con `@Mock` / `@InjectMocks`, garantendo che i test verifichino esclusivamente la logica di business senza accesso al database.

11.4 Casi di test pianificati

11.4.1 `create(BugCreateRequest, UUID)`

ID	Scenario	Risultato atteso
TC-C1	Creazione con titolo, descrizione, tipo e priorità validi	Bug salvato; history CREATE registrata
TC-C2	UserId inesistente nel sistema	<code>EntityNotFoundException</code>
TC-C3	Request con lista di label (alcune esistenti, altre nuove)	Bug con label associate; label inesistenti create automaticamente

11.4.2 `assign(UUID, AssignRequest, UUID)`

ID	Scenario	Risultato atteso
TC-A1	Admin assegna bug a utente esistente	Assignee aggiornato; history ASSIGN registrata
TC-A2	Utente non-admin tenta l'assegnazione	<code>SecurityException</code>
TC-A3	BugId inesistente	<code>EntityNotFoundException</code>

11.4.3 `patch(UUID, BugPatchRequest, UUID, boolean)`

ID	Scenario	Risultato atteso
TC-P1	Admin aggiorna titolo e priorità	Campi aggiornati; history UPDATE registrata
TC-P2	Non-admin tenta di cambiare stato di bug altrui	<code>SecurityException</code>
TC-P3	Assegnatario modifica il proprio bug senza cambiare stato	Modifica applicata con successo

11.5 Criteri di ingresso e uscita

Ingresso: codice sorgente compilabile; dipendenze Maven risolte; ambiente Java 17+ configurato.

Uscita: la suite di unit test dedicata a `BugService` esegue 20 test con **BUILD SUCCESS**, 0 failures e 0 errors; copertura dei rami decisionali principali raggiunta.

11.6 Ambiente di esecuzione

- **JDK:** Java 17 come baseline di progetto; esecuzione verificata anche su Eclipse Temurin 21.0.7 in ambiente containerizzato
- **Build tool:** Apache Maven con plugin Surefire 3.1.2
- **CI locale:** `mvn test` dalla directory `backend/`

12 Strategie di Testing

Questa sezione documenta le strategie adottate per la progettazione dei test di unità del servizio **BugService**, nucleo della logica di business dell'applicazione BugBoard26.

12.1 Metodi testati

Sono stati selezionati tre metodi non banali, ciascuno con almeno due parametri, come richiesto dalla specifica di progetto:

1. `create(BugCreateRequest request, UUID userId)` – 2 parametri
2. `assign(UUID bugId, AssignRequest request, UUID userId)` – 3 parametri
3. `patch(UUID id, BugPatchRequest request, UUID userId, boolean isAdmin)` – 4 parametri

12.2 Classi di equivalenza

Per ciascun metodo sono state individuate le seguenti classi di equivalenza.

12.2.1 `create(BugCreateRequest, UUID)`

Classe	Condizione	Validità	Test case
EC-C1	<code>userId</code> esistente; request con campi obbligatori validi, senza label	Valida	TC-C1
EC-C2	<code>userId</code> non presente nel database	Invalida	TC-C2
EC-C3	Request con lista di label non vuota (misto label esistenti e nuove)	Valida	TC-C3

12.2.2 `assign(UUID, AssignRequest, UUID)`

Classe	Condizione	Validità	Test case
EC-A1	Chiamante con ruolo ADMIN ; bug e assegnatario esistenti	Valida	TC-A1
EC-A2	Chiamante con ruolo USER (non admin)	Invalida	TC-A2
EC-A3	<code>bugId</code> non presente nel database	Invalida	TC-A3

12.2.3 patch(UUID, BugPatchRequest, UUID, boolean)

Classe	Condizione	Validità	Test case
EC-P1	<code>isAdmin=true</code> ; modifica di campi generici (titolo, priorità)	Valida	TC-P1
EC-P2	<code>isAdmin=false</code> ; cambio di <code>status</code> su bug assegnato ad altro utente	Invalida	TC-P2
EC-P3	<code>isAdmin=false</code> ; modifica effettuata dall'assegnatario stesso (senza cambio di stato)	Valida	TC-P3

12.3 Analisi dei Valori Limite (Boundary Value Analysis)

L'analisi dei valori limite (BVA) complementa le classi di equivalenza concentrandosi sui confini delle condizioni di input, dove statisticamente si concentrano la maggior parte dei difetti.

12.3.1 create(BugCreateRequest, UUID)

ID	Valore limite	Tipo confine	Risultato atteso
BV-C1	Titolo con 1 carattere (minimo valido)	Limite inferiore	Bug creato con successo
BV-C2	Titolo vuoto (0 caratteri)	Sotto il limite	Eccezione di validazione
BV-C3	Titolo con 255 caratteri (massimo VARCHAR)	Limite superiore	Bug creato con successo
BV-C4	Lista label vuota (<code>size = 0</code>)	Limite inferiore	Bug creato senza label
BV-C5	Lista con 1 label (minimo non vuoto)	Appena sopra il limite	Bug creato con 1 label associata
BV-C6	Descrizione con 1 carattere	Limite inferiore	Bug creato con successo

12.3.2 assign(UUID, AssignRequest, UUID)

ID	Valore limite	Tipo confine	Risultato atteso
BV-A1	Bug con assegnatario <code>null</code> (prima assegnazione)	Limite inferiore	Assignee impostato; history registrata
BV-A2	Bug già assegnato allo stesso utente (riassegnazione identica)	Confine	Assegnazione eseguita (idempotente)
BV-A3	Bug con assegnatario diverso (riassegnazione)	Sopra il limite	Assignee aggiornato; notifica al precedente
BV-A4	<code>assigneeId</code> uguale a UUID nullo (<code>null</code>)	Confine invalido	Eccezione di validazione

12.3.3 patch(UUID, BugPatchRequest, UUID, boolean)

ID	Valore limite	Tipo confine	Risultato atteso
BV-P1	Request con tutti i campi null (nessuna modifica)	Limite inferiore	Bug invariato; nessuna history
BV-P2	Request con un solo campo non null (modifica minima)	Appena sopra il limite	Solo quel campo aggiornato
BV-P3	Request con tutti i campi valorizzati (modifica massima)	Limite superiore	Tutti i campi aggiornati; history completa
BV-P4	isAdmin=false , utente è l'assegnatario, cambio stato da TODO a IN_PROGRESS	Confine autorizzazione	Modifica consentita
BV-P5	isAdmin=false , utente è l'assegnatario, cambio stato da RESOLVED a TODO	Confine stato	Dipende dalla logica di business (transizione potenzialmente non valida)
BV-P6	Priorità cambiata da LOW a URGENT (estremi enum)	Confine enum	Modifica applicata

L'analisi BVA ha evidenziato la necessità di verificare in particolare:

- La gestione dei campi opzionali (**null** vs. valore presente) nel **BugPatchRequest**
- Il comportamento della creazione bug con liste label di dimensione 0 e 1
- Le transizioni di stato ai confini del ciclo di vita del bug
- La riassegnazione idempotente (stesso assegnatario)

12.4 Copertura strutturale

Oltre alle classi di equivalenza, i test sono stati progettati per coprire i rami decisionali principali (*branch coverage*) della logica di business:

- **create:**
 - Ramo: utente trovato vs. utente non trovato (**findById** → **Optional.empty**)
 - Ramo: lista label presente vs. assente
 - Ramo: label già esistente (**findByName** → **present**) vs. label nuova (**findByName** → **empty** → **save**)
- **assign:**
 - Ramo: bug trovato vs. bug non trovato
 - Ramo: chiamante admin vs. non-admin (controllo **role == ADMIN**)
- **patch:**
 - Ramo: **isAdmin == true** → modifica consentita
 - Ramo: **isAdmin == false** && cambio status && utente ≠ assegnatario → **SecurityException**
 - Ramo: **isAdmin == false** && utente == assegnatario → modifica consentita
 - Ramo: campi **null** nel **BugPatchRequest** → nessuna modifica per quel campo

12.5 Tecnica di isolamento: Mock Objects

Per isolare il *System Under Test* (SUT) dalle dipendenze esterne, tutti i repository JPA sono sostituiti da mock Mockito:

Dipendenza	Mock
BugRepository	@Mock
UserRepository	@Mock
CommentRepository	@Mock
LabelRepository	@Mock
HistoryRepository	@Mock

Questo approccio garantisce:

- **Determinismo**: i test non dipendono dallo stato di un database reale; ogni esecuzione produce lo stesso risultato.
- **Velocità**: nessun I/O di rete o disco; la suite resta comunque eseguibile in pochi secondi anche in ambiente containerizzato.
- **Granularità**: ogni test verifica un singolo comportamento della logica di servizio, facilitando la localizzazione dei difetti.

12.6 Riepilogo dei risultati

L'esecuzione della suite produce il seguente esito:

```
Tests run: 20, Failures: 0, Errors: 0, Skipped: 0  
BUILD SUCCESS
```

Tutti i 20 test della suite dedicata a **BugService** passano con successo, confermando il corretto funzionamento della logica di business per le tre funzionalità testate.

13 Report di Qualità del Codice (Backend)

13.1 Strumenti di analisi utilizzati

L'analisi della qualità del codice del backend è stata condotta utilizzando:

- **SonarQube 10 Community Edition** eseguito localmente tramite container Docker, con task registrato in `backend/target/sonar/report-task.txt`
- **JaCoCo** per la generazione automatica del report di coverage in `backend/target/site/jacoco/`
- **SonarLint 10.x** integrato in IntelliJ IDEA come supporto durante lo sviluppo
- **JUnit 5 + Mockito 5** per la verifica della copertura funzionale dei casi di business principali
- Revisione manuale del codice (*code review*)

13.2 Dashboard di qualità

La seguente tabella riassume le metriche principali emerse dal report SonarQube del backend, integrate con la copertura prodotta da JaCoCo.

Metrica	Valore	Rating
Quality Gate	—	Passed
Bugs	0	A
Vulnerabilities	2	B
Code Smells	24	A
Security Hotspots	4	B
Duplications	0.0%	A
Technical Debt	contenuto	A
Lines of Code (NCLOC)	1855	—
Test Coverage (JaCoCo)	22.0%	—

Rating SonarQube: A = ottimo, B = buono, C = accettabile, D = scarso, E = critico. Il Quality Gate “Passed” indica che il progetto soddisfa le soglie minime di qualità. Il report analitico dedicato è allegato anche nel file `SonarQube_Report.md`.

13.3 Metriche di dimensione

Metrica	Valore
File sorgente Java	48
Linee di codice Java	2 547
Classi	26
Interfacce (Repository)	6
Enumerazioni	5
Record (DTO)	11
Package	8
File di test backend	6
Casi di test backend	32

13.4 Distribuzione del codice per package

Package	File	LOC	Contenuto
<code>model</code>	12	663	Entità JPA, embeddable e 5 enumerazioni
<code>service</code>	7	932	Business logic, export, notifiche, schedulazione e specifiche
<code>controller</code>	6	465	Endpoint REST e gestione centralizzata degli errori
<code>dto</code>	11	117	Record per request/response e metriche
<code>repository</code>	6	69	Interfacce Spring Data JPA
<code>security</code>	2	122	JWT service e filter
<code>config</code>	2	162	Security config e seed
<code>exception</code>	1	6	Eccezioni applicative dedicate
<code>root</code>	1	11	Entry point

13.5 Issue rilevate (dettaglio)

13.5.1 Bugs (0 issue — Rating A)

Nessun bug rilevato. I due bug precedentemente segnalati (UUID hard-coded in `getCurrentUserId()` e `isCurrentUserAdmin()` che ritornava sempre `true`) sono stati corretti: entrambi i metodi ora estraggono l'identità dell'utente autenticato direttamente dal token JWT tramite il `JwtAuthFilter`.

13.5.2 Code Smells (24 issue — Rating A)

Tra i 24 code smell rilevati dal report, i seguenti risultano i più rappresentativi per impatto sulla manutenibilità:

#	File	Regola SonarLint	Sev.	Descrizione
CS-1	BugService	S3776 (Cognitive Complexity)	Major	<code>patch()</code> ha complessità cognitiva 18 (soglia: 15)
CS-2	BugService	S1488 (Local var)	Minor	Lookup utente duplicato in 4 metodi
CS-3	AnalyticsService	S6204 (Stream)	Major	<code>getMetrics()</code> carica tutti i bug in memoria
CS-4	AnalyticsService	S6204 (Stream)	Major	<code>getMonthlyReport()</code> carica tutti i bug in memoria
CS-5	BugSpecification	S1118 (Utility class)	Minor	Manca costruttore privato

13.5.3 Security Hotspot (4 issue — Rating B)

I security hotspot rilevati riguardano soprattutto la configurazione CORS per gli ambienti di sviluppo, la gestione dei cookie JWT e alcuni punti sensibili da riesaminare in vista di un deployment produttivo. Nel contesto del progetto didattico risultano circoscritti e documentati, ma non possono essere descritti come assenti.

13.6 Duplicazioni

Il tasso di duplicazione del codice è pari allo 0.0%, segnale di una buona separazione delle responsabilità e di assenza di porzioni copiate in modo sistematico.

13.7 Punti di forza

- Nessun bug bloccante rilevato dall'analisi statica
- Architettura pulita con separazione netta tra layer
- DTO immutabili implementati come record Java
- Suite di test backend articolata su 6 file di test e 32 casi, con copertura mirata della business logic e dei flussi di sicurezza
- Technical debt contenuto (~1h)

13.8 Riepilogo Quality Gate

Il progetto supera il Quality Gate con un quadro complessivamente positivo e con criticità concentrate in un numero limitato di aspetti da monitorare:

- Bugs: 0 (soglia: 0) ✓
- Vulnerabilità: 2, da considerare in un eventuale hardening produttivo
- Security Hotspots: 4, da riesaminare in fase di deployment ✓
- Code Smells: 24, Rating A ✓
- Duplicazioni: 0.0% ✓
- Coverage JaCoCo: 22.0%
- Technical Debt: ~1h ✓

14 Valutazione dell'Usabilità

14.1 Metodologia

La valutazione dell'usabilità di BugBoard26 è stata condotta seguendo un approccio misto che combina:

1. **Test con utenti** — osservazione diretta di 5 partecipanti durante l'esecuzione di task rappresentativi.
2. **Questionario SUS** (System Usability Scale) — somministrato al termine di ogni sessione.
3. **Ispezione euristica** — analisi condotta dal team di sviluppo sulla base delle 10 euristiche di Nielsen.

14.1.1 Partecipanti

ID	Profilo	Esperienza IT	Familiarità c
P1	Studiante Informatica (3° anno)	2 anni di programmazione universitaria	Bassa (GitHub)
P2	Studiante Informatica con progetti di gruppo	2 anni tra corsi e progetti accademici	Media (GitHub)
P3	Studiante Ingegneria Informatica	1 anno e mezzo di laboratorio	Bassa (GitHub)
P4	Studiante magistrale area software	3 anni tra triennale e magistrale	Media (GitHub)
P5	Tirocinante sviluppo web	2 anni tra studio e progetti personali	Media (Trello,

14.1.2 Ambiente di test

- Browser: Google Chrome 124, risoluzione 1920×1080
- Backend avviato localmente con profilo H2 (dati di esempio precaricati)
- Ogni sessione registrata con screen recording e think-aloud protocol
- Durata media sessione: ~25 minuti

14.1.3 Protocollo

Ogni partecipante ha ricevuto una breve introduzione al sistema (2 minuti) senza dimostrazioni pratiche, quindi ha eseguito in autonomia 7 task predefiniti. Al termine, ha compilato il questionario SUS su carta.

14.2 Risultati dei Task

Task	Successo	% Successo	Tempo medio	Errori medi
T1 — Login con credenziali valide	5/5	100%	12s	0.0
T2 — Creazione nuovo bug con label	5/5	100%	48s	0.2
T3 — Filtro bug per priorità “Alta”	5/5	100%	15s	0.0
T4 — Assegnazione bug (con suggerimento)	4/5	80%	35s	0.6
T5 — Cambio stato bug a “In Progress”	5/5	100%	18s	0.2
T6 — Aggiunta commento a un bug	5/5	100%	22s	0.0
T7 — Archiviazione di un bug risolto	4/5	80%	28s	0.4
Media complessiva		94.3%	25.4s	0.2

Note sui task falliti:

- **T4** — P3 non ha individuato immediatamente il pulsante “Assegna” nella barra azioni della vista dettaglio del bug.
- **T7** — P5 ha confuso il pulsante “Archivia” con “Elimina” e ha esitato prima di procedere all'azione.

14.3 Ispezione Euristica (Nielsen)

L'ispezione euristica è stata condotta dal team di sviluppo analizzando l'interfaccia rispetto alle 10 euristiche di Nielsen. Sono state identificate le seguenti violazioni:

ID	Euristica violata	Descrizione	Gravità
HE-1	Visibilità dello stato	Nessun indicatore di caricamento durante il filtro	2
HE-2	Prevenzione errori	Manca conferma prima dell'archiviazione	3
HE-3	Riconoscimento vs ricordo	Le label disponibili non sono suggerite durante la creazione	2
HE-4	Flessibilità ed efficienza	Mancano shortcut da tastiera per azioni frequenti	1
HE-5	Design minimalista	La dashboard mostra troppe informazioni senza raggruppamento	2
HE-6	Aiuto e documentazione	Assenza di tooltip sui pulsanti con sola icona	2

Scala di gravità: 0 = nessun problema, 1 = cosmetico, 2 = minore, 3 = maggiore, 4 = catastrofico.

14.4 Questionario SUS

Il System Usability Scale (SUS) è composto da 10 affermazioni valutate su scala Likert 1–5. I punteggi sono stati calcolati secondo la formula standard: per le domande dispari si sottrae 1 dal punteggio, per le pari si sottrae il punteggio da 5; i contributi vengono sommati e moltiplicati per 2.5.

14.4.1 Risposte individuali

Domanda SUS	P1	P2	P3	P4	P5
Q1 — Userei frequentemente questo sistema	4	5	4	5	4
Q2 — Il sistema è inutilmente complesso	2	1	2	1	2
Q3 — Il sistema è facile da usare	4	5	3	5	4
Q4 — Avrei bisogno di supporto tecnico	2	1	3	1	2
Q5 — Le funzioni sono ben integrate	4	4	4	5	4
Q6 — C'è troppa incoerenza nel sistema	1	1	2	1	2
Q7 — La maggior parte imparerebbe velocemente	5	5	4	5	4
Q8 — Il sistema è macchinoso da usare	2	1	2	1	1
Q9 — Mi sono sentito sicuro nell'usarlo	4	5	3	5	4
Q10 — Ho dovuto imparare molte cose prima	1	1	2	1	2

14.4.2 Calcolo punteggi SUS

Partecipante	Somma contributi	Punteggio SUS	Giudizio
P1	32	80.0	Buono
P2	37	92.5	Eccellente
P3	26	65.0	Sufficiente
P4	38	95.0	Eccellente
P5	31	77.5	Buono
Media		82.0	Buono

Il punteggio SUS medio di **82.0** colloca BugBoard26 al di sopra della media di riferimento (68) e nella fascia “Buono” (grado B) secondo la scala di Bangor et al. (2009). Il sistema risulta particolarmente apprezzato da utenti con esperienza pregressa su bug tracker (P2, P4), mentre l'utente meno esperto (P3) ha ottenuto il punteggio più basso, suggerendo margini di miglioramento nell'onboarding.

14.5 Feedback qualitativo (Debriefing)

Al termine di ogni sessione è stato condotto un breve debriefing semi-strutturato. I temi ricorrenti sono:

- **Punti di forza citati:**

- Interfaccia pulita e moderna (citato da 5/5 partecipanti)
- Suggerimento automatico dell'assegnatario molto utile (4/5)
- Filtri e ricerca intuitivi (4/5)
- Visualizzazione cronologia/history chiara (3/5)

- **Criticità segnalate:**

- Manca una conferma esplicita prima dell'archiviazione (3/5)
- Le icone dei pulsanti non sono sempre auto-esplicative (2/5)
- Il campo “deadline” non mostra un calendario visuale (2/5)
- Non è chiaro come annullare un'assegnazione (1/5)

14.6 Azioni correttive

Sulla base dei risultati della valutazione, sono state identificate le seguenti azioni correttive, ordinate per priorità:

ID	Problema	Azione proposta	Priorità
AC-1	Archiviazione senza conferma (HE-2)	Aggiungere dialog di conferma	Alta
AC-2	Icone non auto-esplicative (HE-6)	Aggiungere tooltip su tutti i pulsanti	Media
AC-3	Label non suggerite (HE-3)	Implementare autocomplete per le label	Media
AC-4	Indicatore caricamento filtri (HE-1)	Aggiungere skeleton/spinner durante il filtro	Bassa
AC-5	Deadline senza calendario (feedback)	Integrare date picker visuale	Bassa

Le azioni AC-1 e AC-2 non risultano ancora implementate nella release corrente e restano priorità esplicite per una successiva iterazione di miglioramento. Le restanti sono pianificate per iterazioni future.